

Machine Learning Project

From-Scratch Implementation and
Statistical Analysis of California
Housing Prices

including some theoretical explanations

Done by: **Mohamed Boukhames**
As a personal project

July 2026

1- Introduction.....	4
1.1- Project Context.....	4
1.2- Objectives.....	4
1.3- Project Overview.....	5
1.4- Implementation Approach.....	6
2- Exploratory Data Analysis.....	6
2.1- Dataset Overview.....	6
2.2- Feature Distributions.....	8
2.3- Outlier Analysis.....	12
2.4- Geographic Distribution.....	15
2.5- Correlation Analysis.....	16
2.6- Feature–Target Relationships.....	18
3- Data Cleaning and Preprocessing.....	20
3.1- Categorical Variable Encoding.....	20
3.2- Missing Values Analysis.....	22
4- Feature Engineering and Transformation.....	23
4.1- Motivation.....	23
4.2- New Features.....	24
4.3- Logarithmic Transformations.....	26
4.4- Feature Scaling.....	28
4.5- Validation Set Approach.....	29
5. Regression Analysis.....	30
5.1- Linear Regression.....	30
5.1.1- Theoretical Background.....	30
5.1.2- Estimating the Coefficients.....	31
5.1.3- Model Evaluation.....	34
5.1.4- Statistical Inference.....	36
5.1.5- Residual Analysis.....	38
5.1.6- Subset Selection.....	40
5.2- K-Nearest Neighbors Regression.....	44
5.2.1- Theoretical Background.....	44
5.2.2- Choice of K.....	44
5.2.3- Comparing with Linear Regression.....	46
6. Binary Classification Analysis.....	47
6.1- Theoretical Background.....	47
6.2- Cost Function.....	49
6.3- Gradient Descent.....	50

6.4- Logistic Regression Coefficients.....	55
6.5- Assessing the Accuracy.....	57
7. Multi-Class Classification Analysis.....	57
7.1- K-Nearest Neighbors Classification.....	58
7.1.1- Theoretical Background.....	58
7.1.2- Choice of K.....	59
7.2- Generative Classification Models.....	60
7.2.1- Theoretical Background.....	60
7.2.2- Estimation of $P(Y)$	61
7.2.3- Linear Discriminant Analysis.....	62
7.2.4- Quadratic Discriminant Analysis.....	64
7.2.5- Naive Bayes.....	65
8. Improvements.....	66
9. Conclusion.....	67
10. References.....	68
11. Appendices.....	69
11.1- Appendix A.....	69
11.2- Appendix B.....	69

1- Introduction

1.1- Project Context

A lot of people find themselves wanting to purchase a house with some **specific properties** (*like the number of bedrooms, ocean proximity or the location*), but before starting to look for the desired house they want to **predict its price** to determine if it's suitable for their budgets.

In this project, some **Machine Learning approaches** are used to give these predictions.

The full project can be found [here](#).

1.2- Objectives

The aim of this project is to give **good** predictions regarding the price of a house from certain properties (*features / predictors*). In this project, this is done by:

- Finding a good training dataset
- Exploring and understanding the used dataset
- Preparing the dataset for training and engineering some new features
- Using several ML algorithms
- Comparing the models and evaluating their predictive performance

1.3- Project Overview

In this project, the following pipeline was followed:

A. Data Exploration:

- Overviewing and analyzing the features
- Handling missing values
- Visualizing the dataset

B. Data Cleaning:

- Encoding non-numerical features

C. Feature Engineering:

- Engineering new features
- Modifying skewed features
- Scaling features
- Preparing the dataset for classification

D. Regression:

- Fitting a linear model and assessing its accuracy
- Performing subset selection
- Using KNN regression

E. Classification:

- Using binary logistic regression and gradient descent
- Using KNN classification

- Using generative classification models (LDA, QDA and Naive Bayes)

1.4- Implementation Approach

This project was implemented using **Python**, and did **not** use the scikit-learn library or any other machine learning library as all algorithms were implemented **manually** from scratch. This choice was made for educational purposes.

2- Exploratory Data Analysis

2.1- Dataset Overview

The used dataset in this project was

California Housing Prices

The data contains information about houses found in **California districts** and some summary statistics about them based on the 1990 California census.

The dataset contains **20640 observations (observed districts)** with around **1% containing null values in one of the columns.**

The dataset contains **9 features**, which are:

Feature	Type	Range	Mean	Description
---------	------	-------	------	-------------

longitude	Numerical Continuous	[-124.35, -114.31]	-119.569704	Measures how far west a district is
latitude	Numerical Continuous	[32.54, 41.95]	35.631861	Measures how far north a district is
housing_median_age	Numerical Discrete	[1, 52]	28.639486	The median age of houses within the district <i>This feature is capped at 52</i>
total_rooms	Numerical Discrete	[2, 39320]	2635.763081	The total number of rooms across all houses in the district
total_bedrooms	Numerical Discrete	[1, 6445]	537.870553	The total number of bedrooms across all houses in the district
population	Numerical Discrete	[3, 35682]	1425.476744	The total number of residents
households	Numerical Discrete	[1, 6082]	499.53968	The total number of households
median_income	Numerical Continuous	[0.4999, 15]	3.870671	The median income for households <i>It is expressed in tens of thousands of US Dollars</i>
ocean_proximity	Categorical	{<1H OCEAN, INLAND, NEAR OCEAN, NEAR BAY, ISLAND}	Mode= <1H OCEAN	Descriptor of how close the district is to the ocean or ocean bay

- Location Features
- Demographic Features
- Financial Features

The **target variable** is '**median_house_value**', which represents the median value of homes in the block group (*in \$*).

Note: This feature is capped at \$500,001. Homes worth more than \$500,000 are recorded as \$500,001

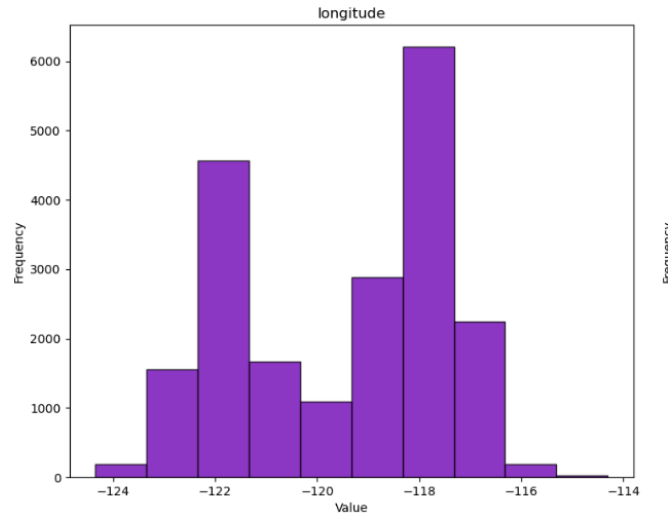
The **target variable** ranges between \$14,999 and \$500,001, and has a mean of $\approx$ \$206,856

2.2- Feature Distributions

In this section, I will discuss the distributions of the feature variables along with the target variable. **Histograms** are used as visual representations.

The histograms that are not present in this section can be found in the appendix A

A. longitude:



It is obvious that the distribution shape is **Bimodal**, having 2 peaks at around **-122 and -118** meaning that a lot of districts fall in this region. Of course, these 2 peaks are **not** random as we have:

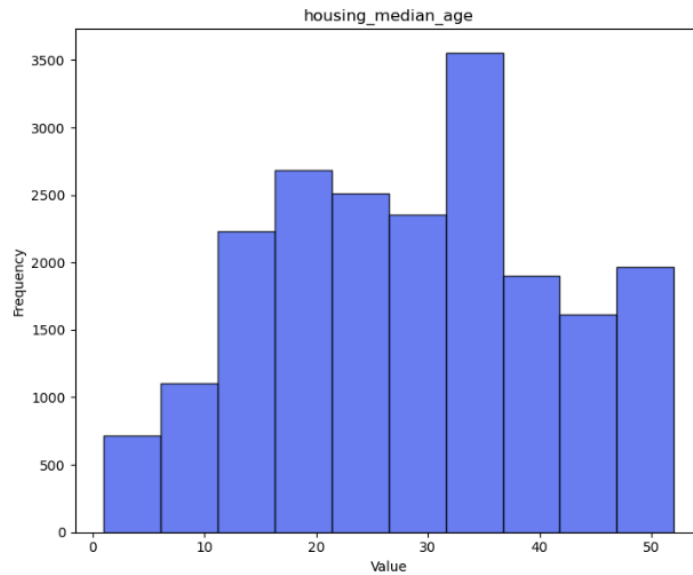
- Near -122: **The San Francisco Bay Area**
- Near -118: **Greater Los Angeles Area**

which are major population clusters.

B. latitude:

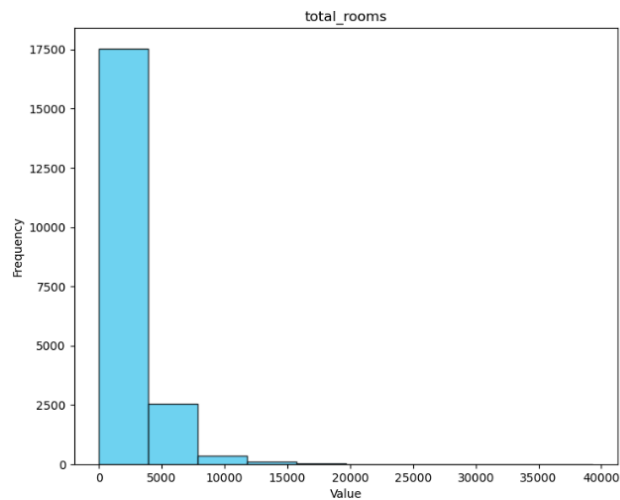
The distribution shape is **Bimodal** as well, with 2 peaks at around **34 and 38**. The reason for these 2 peaks becomes **very obvious** when looking back at the longitude's distribution peaks (*-122 and -118*), since the major population clusters **The San Francisco Bay Area** and **Greater Los Angeles Area** fall in **(38 North, -122 West)** and **(34 North, -118 West)** respectively which corresponds to the 4 peaks of the longitude and latitude.

C. housing_median_age:



The distribution is **fairly uniform** having a spike at around **35**, and **low frequencies** for **new houses** (from 1 to 10 years). It also has an **artificial spike** at 50 because of the capped values.

D. total_rooms:



It is **strongly right-skewed** with most districts having **less than 5,000 rooms** and some districts having **up to 40,000 rooms** (*rare high density districts*).

This is why a long tail extending toward 40,000 is found.

E. total_bedrooms:

The shape is **strongly right-skewed** as well, which is a natural follow-up from the previous result since bedrooms are a type of rooms.

Most districts having **less than 1000 bedrooms** with a long right tail extending to **6000**

F. population:

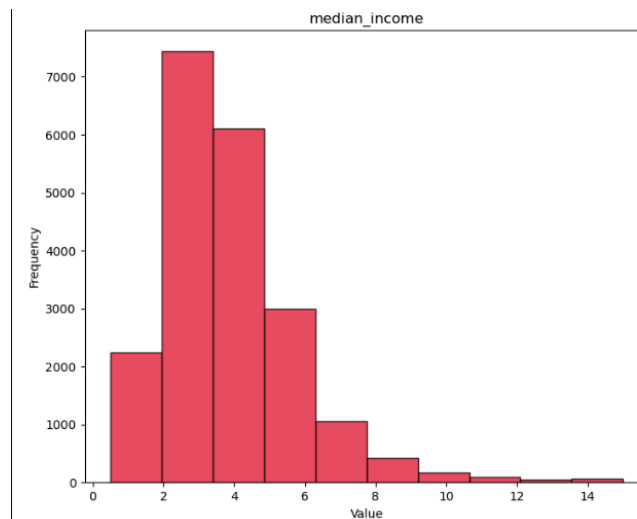
It is **extremely right-skewed** for the same previous reason of *rare high density districts that have more rooms, more bedrooms and a higher population.*

It is concentrated **below 3000** per district, declining toward higher population counts, reaching up to **35,000**

G. households:

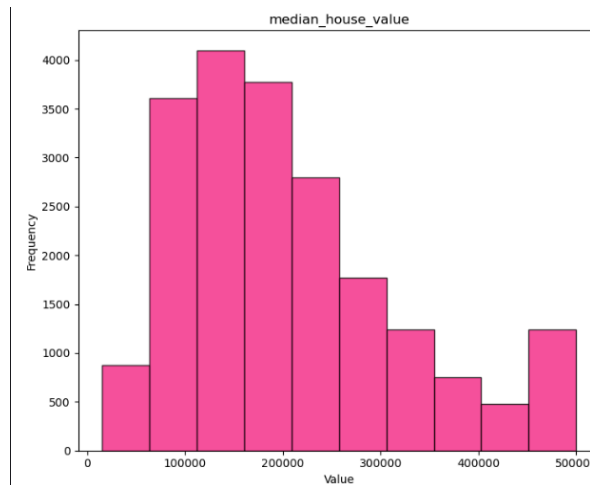
It is **right-skewed**. Concentrated below 1000 households per district stretching out to over **6000**

H. median_income:



The distribution is **moderately right skewed**, it looks like a **bell curve** centered at **3 (\$30,000)** and having a right tail going up to **15 (\$150,000)** for high income districts.

I. median_house_value (target variable):



The shape is **unimodal** and **right-skewed** with an **artificial spike at \$500,001** for the capped values.

Peak house values lay between **\$100,000 and \$200,000**.

2.3- Outlier Analysis

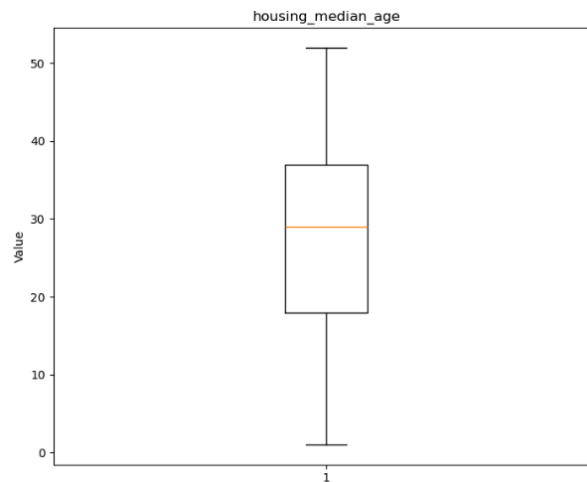
In this section, I will discuss the outliers of the feature variables along with the target variable. **Box Plots** are used as visual representations.

The box plots that are not present in this section can be found in the appendix A

A. longitude & latitude:

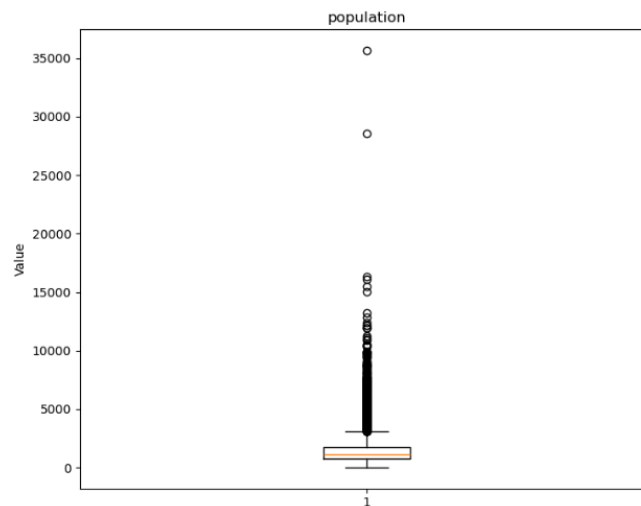
Both features have **no outliers**, this is expected since the data is obtained from an area that is **constrained by geographical boundaries**.

B. housing_median_age:



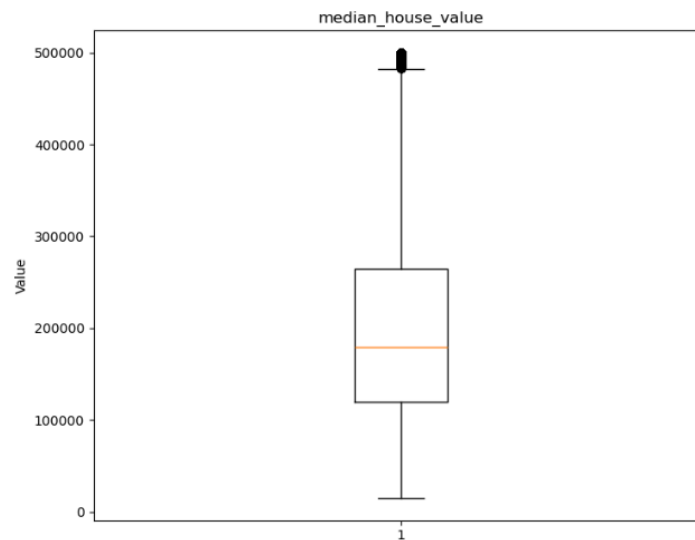
No outliers are found, since this feature is **bounded** by a lower bound of 1 (*cannot have a zero or a negative age*) and an upper bound of 52 *since the values are capped*.

C. total_rooms & total_bedrooms & population & households & median_income:



All of these features have **outliers**, which we already foresaw and explained by analyzing the right skewness of these features. As these features may grow indefinitely and unexpectedly in *high density districts*

D. median_house_value (target variable):



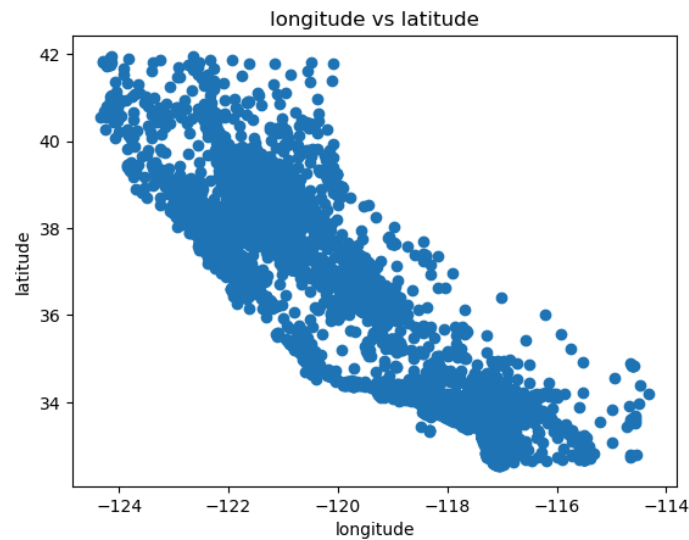
There are **outliers** representing unusually expensive houses (*which are ordinarily found in the same high density districts*).

We could have found more outliers if the values were not capped at \$500,001

The **IQR** for the target variable is [**\$119600, \$264725**]

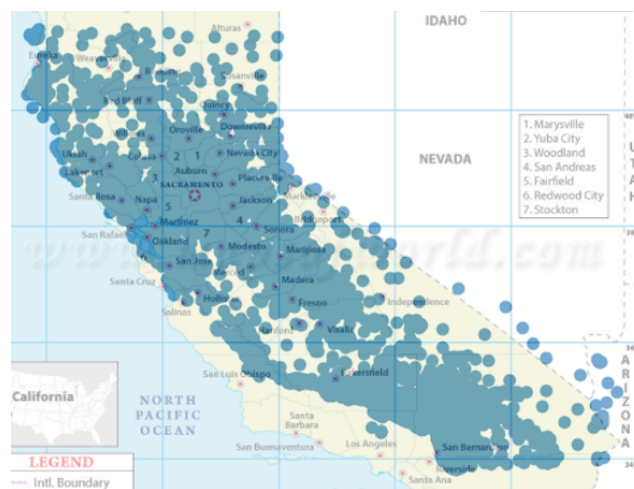
*Meaning that half of the observations lies within this **range**.*

2.4- Geographic Distribution



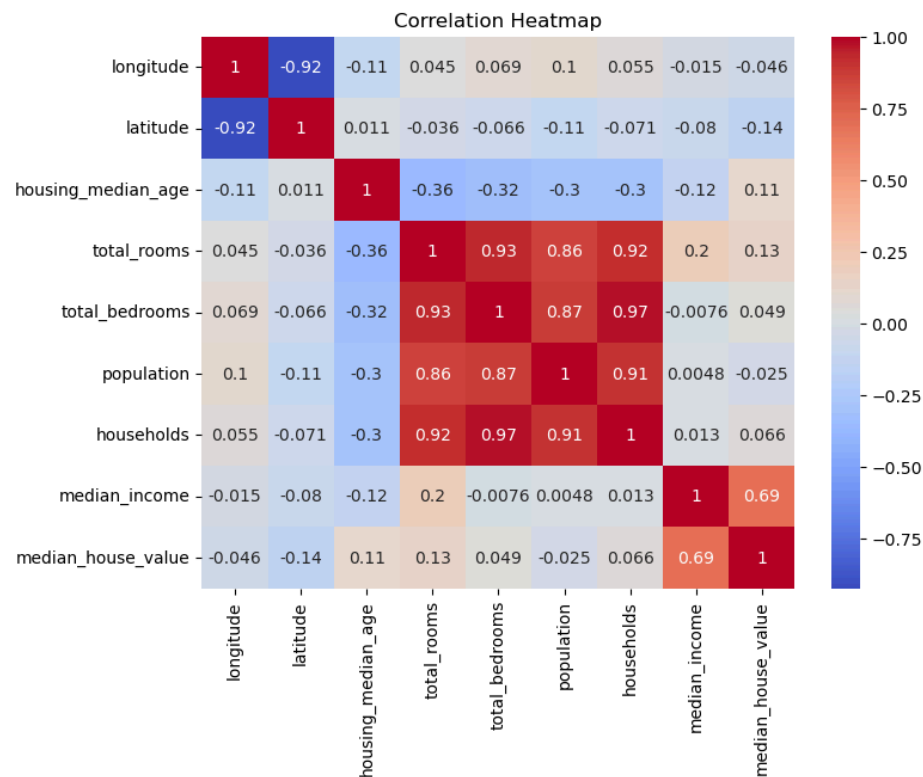
This figure represents a scatter plot for the **longitude** and **latitude**, we can easily see the **inverse relationship between them** (*longitude increases as latitude decreases and vice versa*)

This result is a **natural consequence** of the shape of California, this is better seen by *comparing the Californian map and the scatter plot of our dataset as shown in the next figure.*



The Californian map is obtained from [here](#)

2.5- Correlation Analysis



By looking at the Correlation Heatmap, we can find many features that are correlated with each other, we will look at every *significant* correlation and try to justify it.

1. longitude & latitude:

These 2 features have a **strong negative correlation** ($r=-0.92$).

This correlation has already been explained in the previous section, as we said that it is a **natural consequence of the Californian map**

2. total_rooms & total_bedrooms:

These 2 features have a **strong positive correlation** ($r=0.93$).

This correlation is the most obvious, because bedrooms are also considered as rooms thus **increasing** the number of bedrooms will also **increase** the number of the rooms.

Another reason may be that having more rooms usually implies having more **people** living in the district which would require having **more bedrooms** *for these people to sleep*.

3. population & total_bedrooms:

These 2 features have a **strong positive correlation** ($r=0.87$)

The possible reason for this correlation has already been hinted at in the previous point.

Having a **larger population** implies the need for more **bedrooms** *(more people require more places to sleep at)*.

Additionally, having **more bedrooms** usually means that there is a **larger population**

4. population & total_rooms:

These 2 features have a **strong positive correlation** ($r=0.86$)

The same reason for the correlation between *population* and *total_bedrooms* may be applied in this correlation.

5. median_income & median_house_value:

These 2 features have a **strong positive correlation** ($r=0.69$)

One reason for this correlation may be that expensive houses are usually placed in **good neighborhoods** *(with better schools, low crime rate and proximity to job hubs ...etc)* which naturally attract **high income people**.

This may be the **most interesting** correlation because it involves **the target variable**.

For the other strong correlations, the same reasons and justifications may be applied.

2.6- Feature-Target Relationships

We will not go into *all* the feature-target scatter plots since most of them will not add any extra information.

The features whose relationships with the target variable are **interesting** (*having interesting scatter plots*) are **median_income, longitude and latitude**.

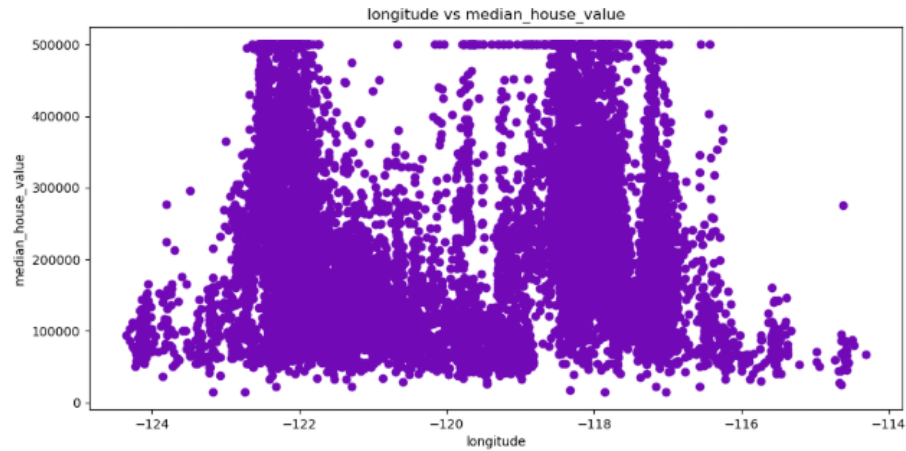
As earlier, the rest of the scatter plots can be found in the appendix A

1) median_income:



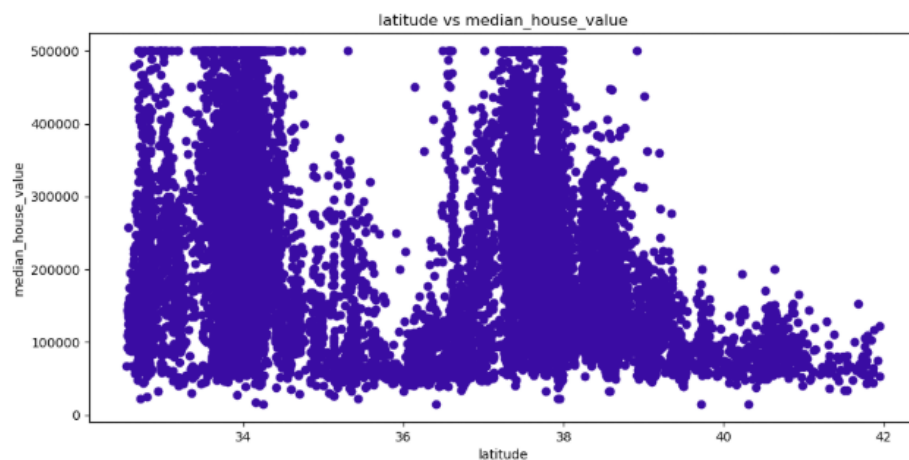
From this scatter plot, we can clearly see that the median_house_value **tends to increase** when the median_income is increasing.

Even though this information is already known from the correlation heatmap studied earlier, this scatter plot allows us to see it more clearly.

2) longitude:

We can see that house prices **spike** at some specific points which are **near -122 and near -118** (*which was justified earlier*).

The prices also drop **significantly** for longitude **less than -123** and **greater than -116**.

3) latitude:

Similar to the longitude, the prices **spike** at **34 and less** and also at **38** and they drop **significantly** for latitude **greater than 38**.

One can conclude that these predictors have strong associations with the target and are therefore potentially useful predictors.

3- Data Cleaning and Preprocessing

3.1- Categorical Variable Encoding

This dataset has one categorical variable which is **'ocean_proximity'**

(it was left aside for most of the analysis done in previous sections)

This feature's unique values fall within a very small set *(containing 5 elements)*:

{'NEAR BAY', '<1H OCEAN', 'INLAND', 'NEAR OCEAN', 'ISLAND'}

Each of these values indicates how close the district is to the ocean or ocean bay

Value	Count
'NEAR BAY'	2290
'<1H OCEAN'	9136
'INLAND'	6551
'NEAR OCEAN'	2658
'ISLAND'	5

This feature is encoded using **one-hot encoding**, by adding a **new feature** for each value of 'ocean_proximity' with **these new features** taking **boolean values (0 or 1)**

*A feature x is **equal to 1** iff the ocean_proximity feature was **equal to x** for that observation, for x in {'NEAR BAY', '<1H OCEAN', 'INLAND', 'NEAR OCEAN', 'ISLAND'}.*

The table below shows all the possible values taken by these new features for every value of ocean_proximity

Value	Feature 1: near bay	Feature 2: <1h ocean	Feature 3: inland	Feature 4: near ocean	Feature 5: island
'NEAR BAY'	1	0	0	0	0
'<1H OCEAN'	0	1	0	0	0
'INLAND'	0	0	1	0	0
'NEAR OCEAN'	0	0	0	1	0
'ISLAND'	0	0	0	0	1

However, following this exact encoding will **yield a critical problem** as the *design matrix's* columns become **linearly dependent**.

The design matrix (*also noted as X*) is just **the dataset** (*of shape $n \times p$, with n being the number of observations and p being the number of features*) with an **extra column containing all 1's** (*1 for all the observations*).

A rule for this design matrix to work correctly with algorithms like *linear regression* is that the columns should be **linearly independent**, but with this used one-hot encoding we can write any of these 5 *new columns* as a linear combination of the *other columns*.

Let's take for example the **near bay** column, this column is completely **determined by the other columns** and can be written as a **linear combination** of them:

$$\text{near bay} = 1 - \text{'<1h ocean'} - \text{inland} - \text{near ocean} - \text{island}$$

With 1 indicating the 1's column

The previous linear dependence can be easily confirmed using the table above.

To fix this problem, a simple solution is to **remove one of these dummy features** *arbitrarily*.

In my case, I decided to remove the feature '*near bay*'.

This will set the **value** 'NEAR BAY' as a reference (or baseline) category, meaning that we indicate that **ocean proximity** = 'NEAR BAY' by setting all the other 4 features (*<1h ocean, inland, near ocean, island*) to 0.

3.2- Missing Values Analysis

As previously mentioned, around **1%** of the observations contain missing (null) values (*more specifically 1.0029%*), which is **207 observations**.

These missing values are all found in **the same feature** which is '**total_bedrooms**'.

The potential strategies for handling these missing values were:

- Deleting the observations containing the missing values:

This strategy may sound reasonable especially since only a small portion (1%) of the observations contain the missing values.

This strategy was **not** used. The missing values are present in **only one feature** thus deleting **207 observations** that have **usable information** in other features is *wasteful*.

- Replace the missing values by the mean of the feature:

This strategy was **not** used. The feature 'total_bedrooms' is **right-skewed** and had **a lot of outliers** (*as it'll be shown later*) which could affect the mean and make it **less reliable**

- Replace the missing values by the median of the feature:

This strategy **was used**. It is the natural *plan b* for not being able to use the mean.

4- Feature Engineering and Transformation

4.1- Motivation

The process of **engineering new features** is very important for the following reasons:

- Our used dataset contains a lot of **aggregate features** like *total_rooms* which give **less insight** than features like *rooms_per_household*.

The difference between these two features is that one tells only *the number of rooms* which can be due to having many houses in the district or to having many rooms per house, whereas the other feature gives better hints to the exact reason

- It makes the **complex relationships** between the features easier to capture, for example if the target relies on *feature A* and *feature B* a linear model can only learn this by adding an **explicit interaction feature** between A and B

Another important process done in this section is **feature transformation** which **changes** the distribution shape of features.

The last process is **feature scaling** which adjusts the **range** of the feature.

4.2- New Features

In this project, **3 features** were engineered. These features are: *rooms_per_household*, *bedrooms_per_room*, *population_per_household*. For each of these features we will discuss **the formula** and **the reason for engineering it**.

1. rooms_per_household

Formula:

$$rooms_per_household = total_rooms / households$$

Reason:

This feature gives insight into **the average number of rooms we find in a household**, this information can be missed without the use of this feature.

For example, a massive district with 500 small apartments may have the **same number of total rooms** as a small neighborhood with 50 mansions. However, it is not possible for these 2 to have the same value in **rooms per household**

2. bedrooms_per_room:

Formula:

$$bedrooms_per_room = total_bedrooms / total_rooms$$

Reason:

This feature captures the **ratio** between the bedrooms and rooms.

Low ratio: indicates that the houses have a lot of *non-bedroom rooms*, for example: dining rooms, bathrooms, ...etc

High ratio: indicates that the houses are full of bedrooms.

3. population_per_household:

Formula:

$$population_per_household = population / households$$

Reason:

Raw ‘population’ does not tell much about the *crowdedness* inside the homes.

4.3- Logarithmic Transformations

As we previously saw many features **suffer from skewness** (*right-skewness specifically*), by applying a logarithmic transformation we can change the distribution shape *to a hopefully bell-like shape*.

A possible question that may arise is ‘*why does logarithmic transformation fix the right skewness?*’ The answer is simple and related to how the logarithmic function behaves for large values.

Right skewness means that there are **rare (low-frequency) big values** (outliers), by applying the logarithmic function these *big values* turn into *smaller values*, and the differences between these outliers and the other data points will **significantly decrease**.

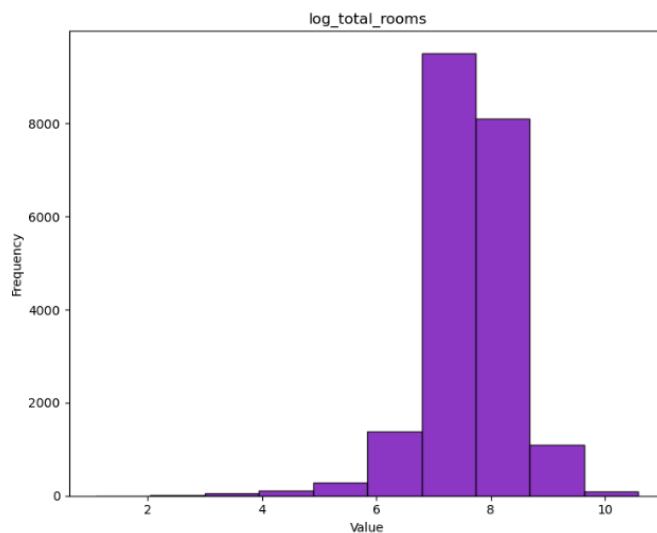
In this project, the used function is:

$$\ln(1+x)$$

The features that were log-transformed are the features that were previously identified as **right-skewed** which are: *'total_rooms', 'total_bedrooms', 'population', 'households'* and *'median_income'*

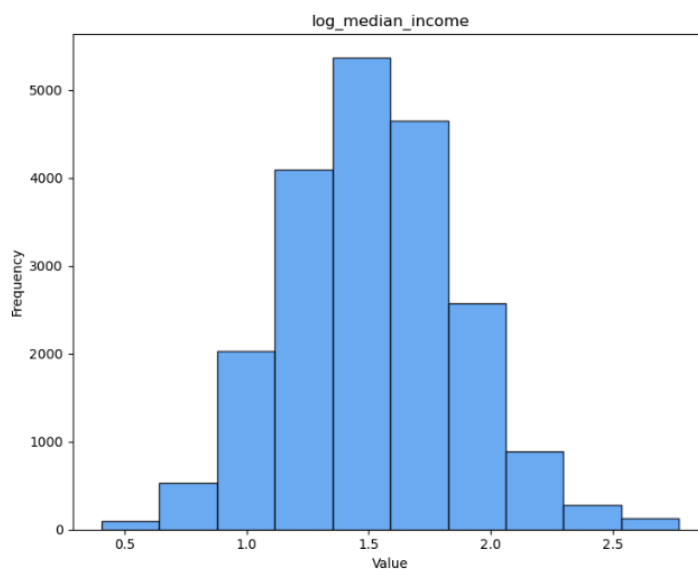
We can see the new distribution shape of these new log-transformed features.

total_rooms:



We can see that the right-skewness **reduced dramatically**, but instead we got a *weaker (but present) left-skewness*

median_income:



After log-transforming *median_income* it turned into a beautiful **bell-like shape** (similar to a normal distribution), because this feature was not **as** skewed as the other features (weaker skewness).

4.4- Feature Scaling

All the original and engineered features except the binary features (were **scaled** using **normal scaling**, which involves the use of the formula: $X_{scaled} = \frac{X - \text{Mean}}{\text{Standard Deviation}}$

By applying feature scaling, we make the distributions **centered at around 0** and have **their standard deviations equal to 1**.

The reasons why **scaling** features is **useful** include:

- It helps algorithms that rely on *distances* like **KNN regression & KNN classification**.

When features are measured in different units, some of them will have **bigger values** (example: *total_rooms* ranges up to 39,000) and other features will have **smaller values** (example: *median_income* ranges from only 0.49 to 15). Thus the *big features* will contribute **much more** in the calculations of the distances (even if they don't really play a big role, the model will miss capturing this).

In other words, if a model needs to look for the **nearest neighbors of a point x**, it will emphasize the data points that have close values to x **in the bigger features**

only, consequently **ignoring** the features with smaller values even if they are *actually informative*.

After scaling, all features will have **values centered at 0** and **not vary away too much** (*since the standard deviations are equal to 1*). Thus all features will have roughly the same range.

4.5- Validation Set Approach

The dataset will be used for two purposes:

- Training:

The main purpose of the dataset is to **train** the models (such as linear regression and KNN regression).

- Testing:

After training the models and preparing them, we wish to **test** the performance of these models on **new unseen data**, this requires testing the models on observations that **were not used during the training process**.

A common choice for being able to **train and test** without the need to gather more observations is to **divide randomly the dataset into: training set** (used for training the models) and **testing set** (contains data points not used during training for testing the model's performance)

In this project, the following division was followed

- Training set: **80%** of the dataset is used for training.
- Testing set: **20%** of the dataset is used for testing.

5. Regression Analysis

5.1- Linear Regression

5.1.1- Theoretical Background

For linear regression, we start by making a **very important assumption** that is:

*The relationship between the target variable Y and the features X_i is **linear***

Meaning:

$$Y = \beta_0 + \sum_{i=1}^{15} \beta_i X_i + \epsilon$$

Y : the target variable 'median_house_value'

β_0 : the bias (or intercept)

β_i : the weight for the feature X_i

X_i : it represents a different feature for each i

ϵ : irreducible error (may come from unmeasured variables, unmeasurable variables...etc)

Although this assumption is most likely **wrong**, linear regression still gives good **inferential** knowledge, meaning that it allows us to tell how much each feature **contributes to** the target variable.

Even if these assumptions are not satisfied, it does not necessarily imply that the linear model will be weak. Even if it will miss the real **non-linear relationship**, it may still give good insights especially for inference. Additionally, the coefficients may become **unstable** if the multicollinearity is strong.

5.1.2- Estimating the Coefficients

The main thing to do in linear regression is to **estimate the coefficients** (*the bias and the weights*), the common choice in linear regression is **Least Squares Estimation**.

LSE aims to estimate the coefficients by **minimizing a specific cost function** (*that represents how far off the linear model is*).

This cost function is **Residual Sum of Squares (RSS)**, which is the sum of squared vertical distances (*also called residuals*) between the actual observed data points and the values predicted by the model line.

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

n : number of observations used for training

y_i : the target value for observation i

$\hat{y}_i$: the predicted value for observation i , which is equal to: $\hat{y}_i = \beta_0 + \sum_{k=1}^{15} \beta_k x_{i,k}$

More simply, the goal is to make **the linear model as close as possible to the data points.**

This cost function is **convex**, meaning that it only has **one minimum** and our goal is to find the coefficients that minimize this function. Thankfully, there is a **closed-form solution**:

$$\hat{\beta} = (X^T X)^{-1} X^T Y$$

$\hat{\beta}$: the coefficients vector

X : the **design** matrix (the observations matrix with an extra column of 1's)

Y : the targets vector

The following table shows the **least squares estimates of the coefficients** for the **unscaled** features along with some insights

Feature	Corresponding Weight (in \$1,000)	Insight
<i>bias</i>	-2530.31	\$2.53 Million baseline that exists to balance out the large positive
longitude	-29.32	Moving 1 Degrees East reduces home value by \$29,320
latitude	-29.71	Moving 1 Degrees North reduces home value by \$29,710
housing_median_age	1.08	Additional year of median neighborhood age adds \$1,080 to home value

bedrooms_per_room	157.64	Warning of a blown up coefficient
population_per_household	1.23	Having more people per household has minor effect
rooms_per_household	2.68	Each additional room per household adds \$2,680 to home value
log_total_rooms	-10.97	Warning, having more rooms should increase the value of a house and not decrease it
log_total_bedrooms	17.82	Adds \$17,820\$ per log-unit
log_population	-107.86	Warning of a blown up coefficient
log_households	-105.8	Warning of a blown up coefficient
log_median_income	203.24	Neighborhood wealth dominates house prices, this feature is the biggest driver of the house values
<1h ocean	10.14	Being 1 hour away from the ocean adds \$10,140 more to the value over the baseline of being near bay
inland	-23.55	Being inland gives \$23,550 drop in value relative to near bay houses
near ocean	0.087	A house being near ocean or near bay is similar (less than \$100 difference)
island	134.75	This coefficient is blown up because only 5 observations fall in this class

The previous results gave **serious warnings** about our model's performance:

- Multicollinearity:

Many of the features are collinear, this made the model confused and destabilized their coefficients.

- Rare category:

In the original *ocean_proximity* feature, only 5 observations fall in the class of ‘**island**’ which resulted in **overfitting** these data points.

Since these 5 observations had *by chance* high house values. The model supposed that the reason behind the high house values is **this feature**, and that being on an island would add up to **\$134,750** regardless of the other features.

A possible solution is to remove one feature from each highly collinear pair.

5.1.3- Model Evaluation

Now the accuracy of the **linear regression model** is assessed using 2 common metrics:

RSE and R^2

We apply these 2 metrics on 2 different *portions* of the complete dataset which are:

- Training set: Contains the exact same data used in the training of the model
- Testing set: Contains observations that were not used during the training of the model

RSE:

This metric is directly linked to the **cost function** that we were trying to minimize earlier (RSS).

$$RSE = \sqrt{\frac{RSS}{n - p - 1}}$$

RSE: residual sum of squares (the formula was given earlier)

n: number of observations

p : number of features (predictors)

$n - p - 1$: represents the degrees of freedom (subtract $p+1$ since we had to estimate p weights and 1 bias, not subtracting this quantity will not make much of a difference since $n \gg p$)

- Training RSE:

Training RSE = \$69,179.38

This means that, on average, our model's predicted prices will be off by roughly \$69,000 from the actual prices on **the same data used for training**.

This suggests that there is still room for improvement.

- Testing RSE:

Testing RSE = \$71,410.11

This means that, on average, our model's predicted prices will be off by roughly \$71,000 from the actual prices on **new unseen data**.

This result may be used to say that (*roughly speaking*): if a person would ask our model to predict from *any given set of values for the predictors*, the prediction would be around \$71,000 away from the real target.

Although this number may seem a bit large, it is **not too bad** for a linear model trying to predict in a non-linear and collinear environment.

This result also suggests that the model is **not overfitting**, since the testing RSE is only **3.2% higher** than the training RSE. It means that the model actually learned some patterns from the data instead of just following random noise found in the

training set (*if the model was overfitting the training set, then it would perform much worse on unseen data relative to the seen data*).

R^2 :

Also known as **the coefficient of determination**, this metric measures **the proportion of the variance in the target variable that is explained by the model**.

- Training R^2 :

Training $R^2 = 62.9\%$

The model is capturing 62.9% of the total variance in the training set.

- Testing R^2 :

Testing $R^2 = 65.14\%$

The unusual result here is that the model performed **better** on unseen data, this is most likely because of the random split used earlier to divide the dataset into training and testing sets which made the **testing set have more variance in the target variable than the training set**.

The proportion of the explained variance is **good enough** as a *starting point* of a linear model.

5.1.4- Statistical Inference

Now, we are going to see the **p-values** of each predictor.

- p-values:

Now we calculate the p-value for each predictor, the p-value gives the

probability that the results happen by pure random chance (*more formally,*

The p-value is the probability, assuming the null hypothesis is true, of obtaining a test statistic at least as extreme as the one observed).

Most of the p-values were **very close to 0**, which indicates that these features do *indeed* play a significant role in our linear model

However, the feature '**near ocean**' has a p-value equal to **97.5%** which indicates that its coefficient (\$85) is most likely due to **random noise** and that observations that were captured **near bay** (*the baseline class*) are similar to observations captured **near ocean**

Another feature that is found to not be **very statistically significant** is

'**log_total_rooms**' with p-value equals to **15%** (*which is high for standard significance levels ranging from 1% to 5%*), this suggests that the model already gets the *rooms-related* information it needs from other features.

It is worth noting that another possible reason for these high values can be due to the *inflated* coefficients we got earlier, since the **calculation of these p-values is very influenced by the estimated coefficients**. *For example, the estimated 'log_total_rooms' coefficient was found to be very inflated, this could be a direct reason why its corresponding p-value was high.*

5.1.5- Residual Analysis

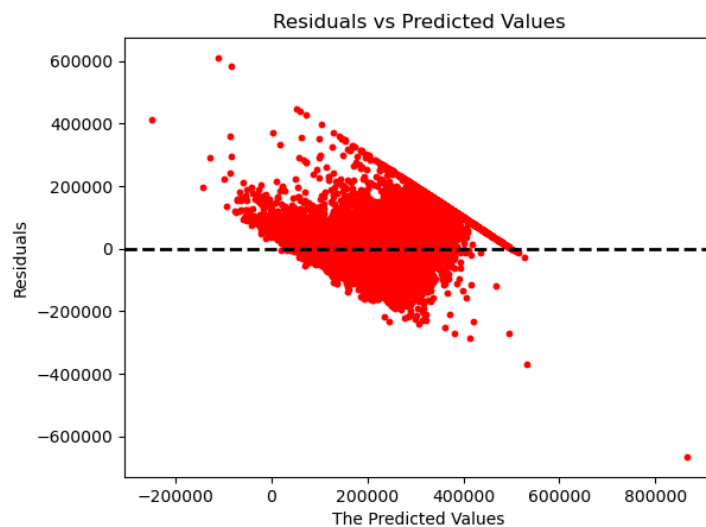
Now, we are going to analyze the residuals (*the vertical difference between the target values and the predicted values by the linear model*)

- Residuals vs Predicted values plot:

This is one of the key plots to analyze residuals, as it helps us check if the linear model has **successfully captured all the possible patterns** in the data, leaving behind only unpredictable noise.

The plot should be **centered at 0**, meaning that the model is not *balancing* between underestimation and overestimation.

The next figure shows this same plot for our project:



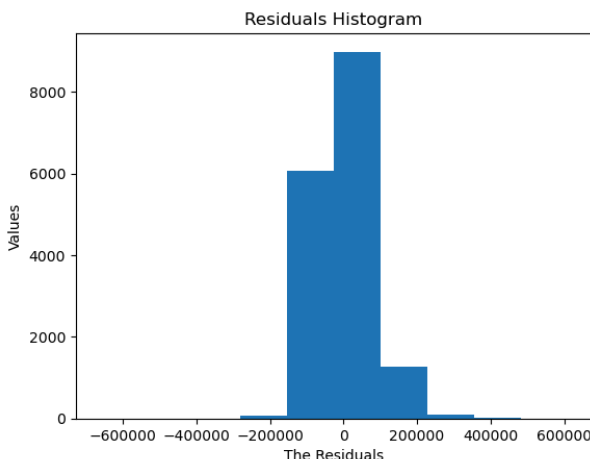
The first thing we notice is that the residuals are **always positive** for small values and then become **always negative** for big values, this indicates that the model is **underestimating for cheap houses** and **overestimating for expensive houses**.

Secondly, we can see a **straight** upper bound which is a consequence of the target variable being capped at \$500,001.

Lastly, we have many **negative predicted values**, because the linear model is *unconstrained (meaning that the predicted prices can be any value in R)*

- Residuals Histogram:

We want the residuals distribution to be ***normal centered at 0***, this is to confirm another assumption (*other than the assumption that the underlying relationship between X and Y is linear*) related to the linear model, which is that: **the errors ϵ are normally distributed with a mean equal to 0**



It is obvious that the residuals are **centered more at the right of 0** (*between \$0 and \$100,000*) which means that the model tends to underestimate more than it overestimates.

Additionally, there are some **large residuals at 600,000 and -600,000**, which means that the model tends to make **extraordinary overestimations and underestimations** (*the outliers are present on the right and on the left*)

5.1.6- Subset Selection

Having a lot of features increases the **complexity** and reduces the **interpretability** of the model that uses them all, subset selection aims to remove features that do not add **much predictive power to the model** for the sake of reducing complexity.

We cannot measure this **power-complexity** tradeoff using RSE or R^2 since they never **worsen** for **any** added feature (*even if it is not really useful*), thus the following metrics are used: Adjusted R^2 , Akaike Information Criterion and Bayesian Information Criterion. All of these metrics **penalize an** increased number of predictors without justification.

In this project, **hybrid subset selection** was used. Hybrid selection works by starting with a model of 0 predictors (*called the null model*), and keeps adding the **best predictors to it** (*measured by one of the previously mentioned metrics*) at the same time it **removes the worst predictors** from the model (*more formally, the predictors whose removal improves the selected criterion*). This process stops when the model cannot be improved by either **adding a new predictor** or **removing from used predictors**. In summary, hybrid subset selection combines both **forward subset selection** and **backward subset selection**.

By applying hybrid selection using **Adjusted R² and AIC**, we noticed that the *same* predictors were added and removed within the same order. The table below shows the steps and the improved metric after each step

Step	Action	Features	Adjusted R ² (aim: maximize)	AIC (aim: minimize)
1	Adding median_income	• median_income	44.88%	7340411565
2	Adding inland	• median_income • inland	55.57%	5916726742
3	Adding housing_median_age	• median_income • inland • housing_median_age	56.82%	5749606348
4	Adding bedrooms_per_room	• median_income • inland • housing_median_age • bedrooms_per_room	57.86%	5612014511
5	Adding <1h ocean	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean	58.21%	5565294147
6	Adding total_bedrooms	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms	58.54%	5522058784
7	Adding population	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population	61.65%	5108462160
8	Adding households	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population • households	62.44%	5002768074
9	Adding population_per_household	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population • households • population_per_household	62.62%	4979971614

10	Adding rooms_per_household	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population • households • population_per_household • rooms_per_household	62.71%	4967210710
11	Adding latitude	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population • households • population_per_household • rooms_per_household • latitude	62.76%	4961868853
12	Adding longitude	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • total_bedrooms • population • households • population_per_household • rooms_per_household • latitude • longitude	64.051%	4789526171
13	Removing total_bedrooms	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • population • households • population_per_household • rooms_per_household • latitude • longitude	64.053%	4789078329
14	Adding island	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • population • households • population_per_household • rooms_per_household • latitude • longitude • island	64.08%	4785183824
15	Adding near ocean	• median_income • inland • housing_median_age • bedrooms_per_room • <1h ocean • population • households • population_per_household • rooms_per_household • latitude • longitude • island • near ocean	64.1%	4783576880

The table above gives a lot of information regarding how much each feature *improved the model at every stage of the process*.

Another useful insight is that *total_bedrooms* lost a lot of *meaning* when a lot of *room-related* features were included.

The feature '*total_rooms*' was **never** added to the model meaning it didn't contribute much in powering up the model **at any point**.

For **Bayesian Information Criterion**, it applies a **stronger** penalty than AIC and Adjusted R^2 for over-complicating the model (*adding more predictors*). Thus, it is more **aggressive** in the selection of the predictors. Using subset selection with BIC, the predictors that were never added were: ***total_rooms* and *near ocean***.

The predictors ***total_rooms* and *total_bedrooms*** were added and then removed, indicating that they became less relevant when more *room-related* predictors were added to the model.

5.2- K-Nearest Neighbors

Regression

5.2.1- Theoretical Background

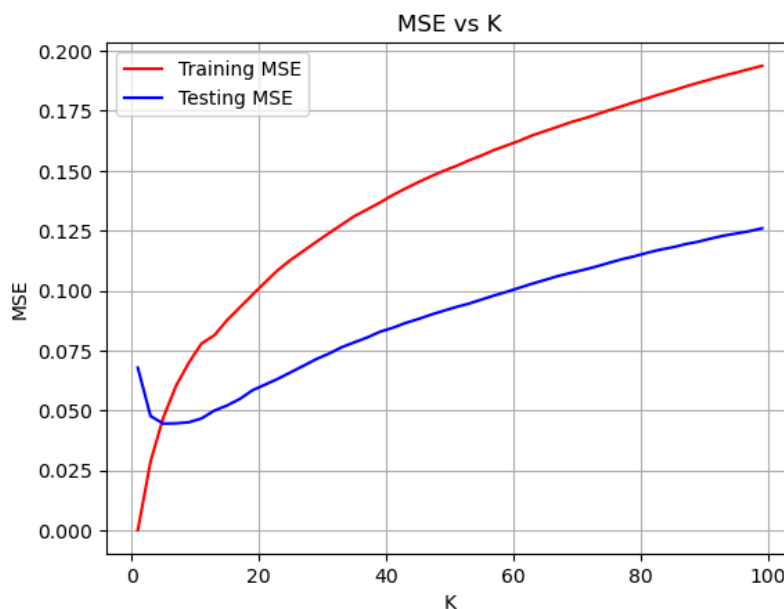
Unlike linear regression where we were trying to fit a model on a training set, then use the same fitted model for any further predictions. KNN regression predicts the target of a **point x** by looking at the **targets of the k closest points to x** and **averaging them**.

For example, if the 3 closest neighbors to x had the targets: 1, 2, 3 then the target of x is predicted to be: $(1+2+3)/3 = 2$.

The *closeness* is measured using the ***Euclidean distance***. Another thing that we need to determine is **the number k** *which represents the number of neighbors we look for when predicting for a given point*.

5.2.2- Choice of K

The choice of k was made by **trying out different values of k** and measuring both the **training and testing MSE**. *(the training set is the dataset used for finding the neighbors)*



The following plot represents the training MSE and testing MSE for k ranging from 1 to 99:

We will analyze both the training MSE and the testing MSE:

Training MSE:

We see that the training MSE is at its best value for $k=1$ where it is **equal to 0**, meaning that for $k=1$ the **model predicted perfectly**. Of course, this is expected. As for $k=1$ we are only looking at **the closest neighbor** when predicting. And when applying this to the training set, predicting the target of a point x is done by only looking at **that same point x** (*since $distance(x,x)=0$, and the points to predict are the same points to predict from*) which will **always have the predicted target equal to the actual target**. This is an obvious example of **overfitting** (*the model is trying too hard to match the training set*).

As k increases, the model keeps performing worse and worse since now it has to look at more data points and averaging them which **reduces the accuracy** from when it was looking at only the same data point.

Testing MSE:

The testing MSE is more interesting to analyze, as now the points we are trying to predict are **different** from the point we are predicting from.

For $k=1$ this time, the model of course cannot predict perfectly. For predicting the target of a point x , it will look at **the closest point to x** *which may or may not be close enough* to yield accurate prediction. The testing MSE takes its minimum at **$k=5$** , which makes it the **chosen value** for the KNN regression model.

As k increases, the model is now looking **at further and further points** and averaging them (*it is treating the furthest point which is the k^{th} closest point in the same priority as the closest point since the classical mean is used*) which **decreases the accuracy**. With this said, we can say that the **best size of the neighborhood to consider is 5**.

5.2.3- Comparing with Linear Regression

In this case, **KNN regression outperformed linear regression**.

But this can be due to the problems that faced linear regression and were **not** addressed *like: multicollinearity*. A problem we might face with KNN regression is that the **training set is finite**, meaning that some points *we want to predict for* will be too **far** away from the training set and even **the closest points** to them *will still be far*. Consequently, after some *bound*, **all the**

points will have **the same closest neighbors** thus the same predicted value no matter how different these points will be.

This issue doesn't happen in linear regression, since the linear model is defined **in an unconstrained way**.

6. Binary Classification

Analysis

In this part, we will move from the regression setting into the **binary classification setting**. This is done by changing the values taken by the target variable to {'**cheap**', '**expensive**'} indicating if the houses in the district tend to be cheap or expensive respectively.

- Houses are considered to be **cheap** if the median value is **less than or equal to \$300,000**
- Houses are considered to be **expensive** if the median value is **greater than \$300,000**

6.1- Theoretical Background

In this setting, we are trying to estimate the **probability distribution** for an observation to be in a *baseline category* (or class). In our project, the baseline category is chosen to be '**expensive**'.

In other words, the model we are trying to estimate is:

$$P(Y = \text{'expensive'} \mid \mathbf{X})$$

Which translates to: The probability of the *house* being **expensive**, *knowing that* the values for the predictors are represented in the *vector* $\mathbf{X}$.

This can be done by making a **crucial assumption** regarding this probability distribution (*similar to linear regression, assuming that the relationship between the predictors and the target is **linear***). Here we assume that the distribution takes the following functional form:

$$P(Y = \text{'expensive'} \mid \mathbf{X}) = \frac{1}{1 + e^{-(\beta_0 + \sum_{i=1}^{15} \beta_i X_i)}}$$

This comes directly from the **sigmoid function** defined by:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

By substituting z by a **linear combination of the predictors**. In other words:

$$P(Y = \text{'expensive'} \mid \mathbf{X}) = \sigma\left(\beta_0 + \sum_{i=1}^{15} \beta_i X_i\right)$$

Then the model will classify a given observation as **'expensive'** if:

$$P(Y = \text{'expensive'} \mid \mathbf{X}=\mathbf{x}) > 1 - P(Y = \text{'expensive'} \mid \mathbf{X}=\mathbf{x})$$

Meaning that, the probability of this observation being expensive is **greater than** the probability of this observation *not being expensive* (i.e. **cheap**). Else, the observation is to be classified as *not expensive* (i.e. **cheap**).

6.2- Cost Function

To fit the linear regression model we used the **RSS as a cost function** (*equivalently RSE or MSE*). Similarly, we will use a cost function to fit the binary logistic model so that we can keep **improving** it (*by minimizing this cost function*) until ***no further improvements can be made*** (that is, our current model is the closest possible to the data point).

In binary logistic regression, the cost function we use is the **log-loss function**.

It measures how ***far*** a predicted probability is from the actual true target ('cheap' or 'expensive'), heavily penalizing ***confident*** wrong answers.

Another point worth mentioning, the log-loss is a **convex function**, meaning that it has only one minimum. That is, there exists only **one vector of values for the parameters β_i** such that this model's log-loss is **lower than all of its neighboring models** (*the models that use slightly different values for the parameters*).

6.3- Gradient Descent

When we were trying to fit the linear regression model earlier, we estimated the **coefficients** (*the bias and the weights of the predictors*) using a **closed-form solution**. Which involved solving one equation derived from *linear algebra*. However, such a solution does **not** exist in logistic regression.

To estimate the **coefficients** β_i in logistic regression, we use **iterative algorithms** that start from a point (*initial values for the predictors*) and find their way to the set of **coefficients** β_i that **minimize** the used cost function..

The algorithm implemented and used in this project is **Gradient Descent**, that starts from a set of initial values taken by the coefficients and finds its way to the minimum using **the gradients** (**the derivatives**), we are sure that the algorithm is always able to find the **global** minimum since the log-loss is a **convex function** *in this setting*.

Gradient descent's algorithm itself is defined by some parameters that we **need to fix** in order to get **as close as possible** to the global minimum, these are:

- Initial values taken by the coefficients: This parameter is not very important, as it does not determine **in any way** how close to the minimum the algorithm is going to end up. It influences *in a small way* how many steps will be needed to get to the minimum, as if the values taken by the coefficients were **initially already close to the minimum** then only a few steps will be needed. However, since initially we don't know anything about where the minimum is located at, the initial values taken by the coefficients **will all be set to 0** (*design choice*).

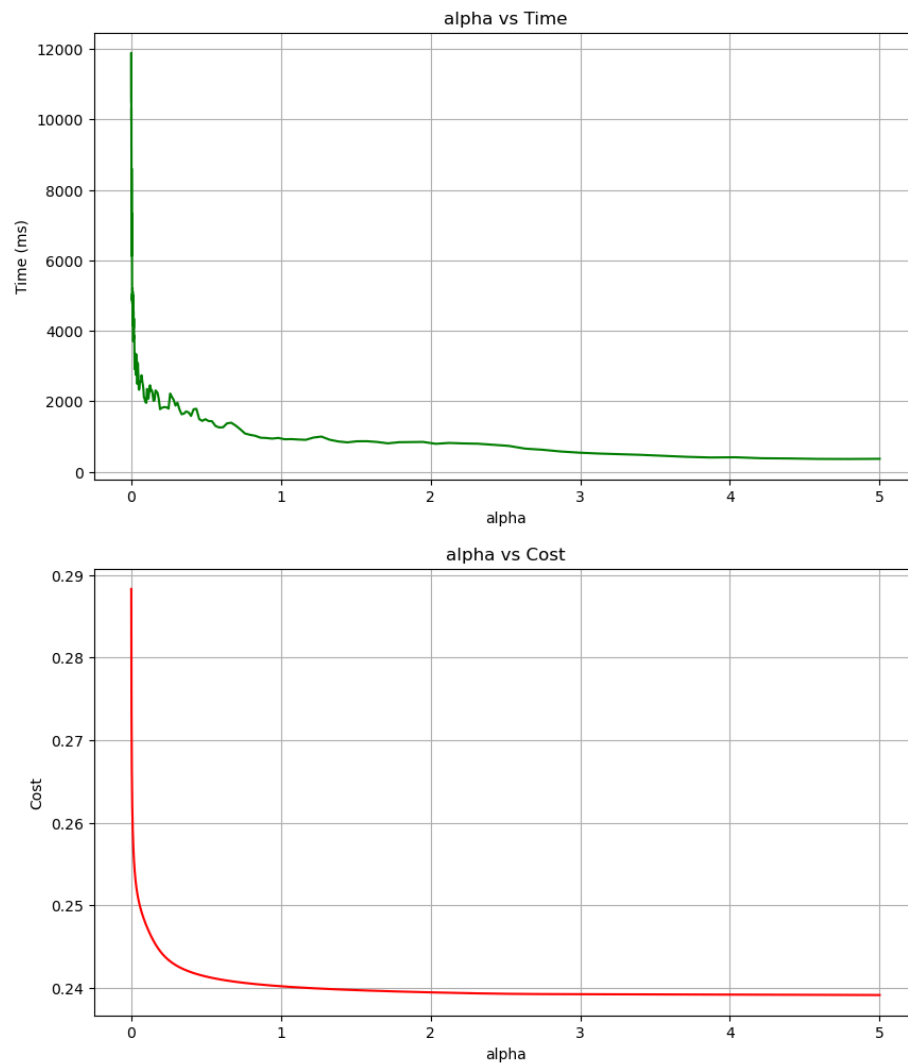
- Learning rate (referred to as α): this parameter *influences (in addition to the gradient at that given point)* the **size** of the step taken by the algorithm. For large learning rates, the algorithm will take **huge steps** so it might reach the minimum in a few **iterations (few steps)**. The risk for very large learning rates is that the algorithm might **miss the global minimum** (*example: the minimum is 4 steps away, but the algorithm takes 10 steps making the algorithm **skip** the minimum and it will be even harder to come back to it since now the minimum is 6 steps away and the algorithm can only move in 10's*). For small learning rates, the algorithm will take **microscopic steps**, meaning that it might end up being very close to the **minimum** but also **very slow**.

Example: algorithm A takes steps of size 10, algorithm B takes steps of size 1 and the closest point to the minimum is 50 steps away. Then, algorithm A will finish in 5 iterations whereas algorithm B will take 50 iterations making the overall process slow.

- Convergence Threshold (referred to as ϵ): this parameter represents **the main stopping condition**, that is **to stop when the difference in the cost is smaller than ϵ** , it is an indicator for **convergence**. Meaning that, the algorithm is no longer improving much of the cost (*for example, the cost is only improving by 0.000001*).

1. Tuning the Learning Rate:

For tuning the learning rate, we first started by **fixing the second parameter that is the convergence threshold** to the value: 10^{-6} (0.000001). And running many gradient descent algorithms with different values for α in the range: **[0.001, 5]**. The following results were given: *(the detailed values can be found in the appendix B)*



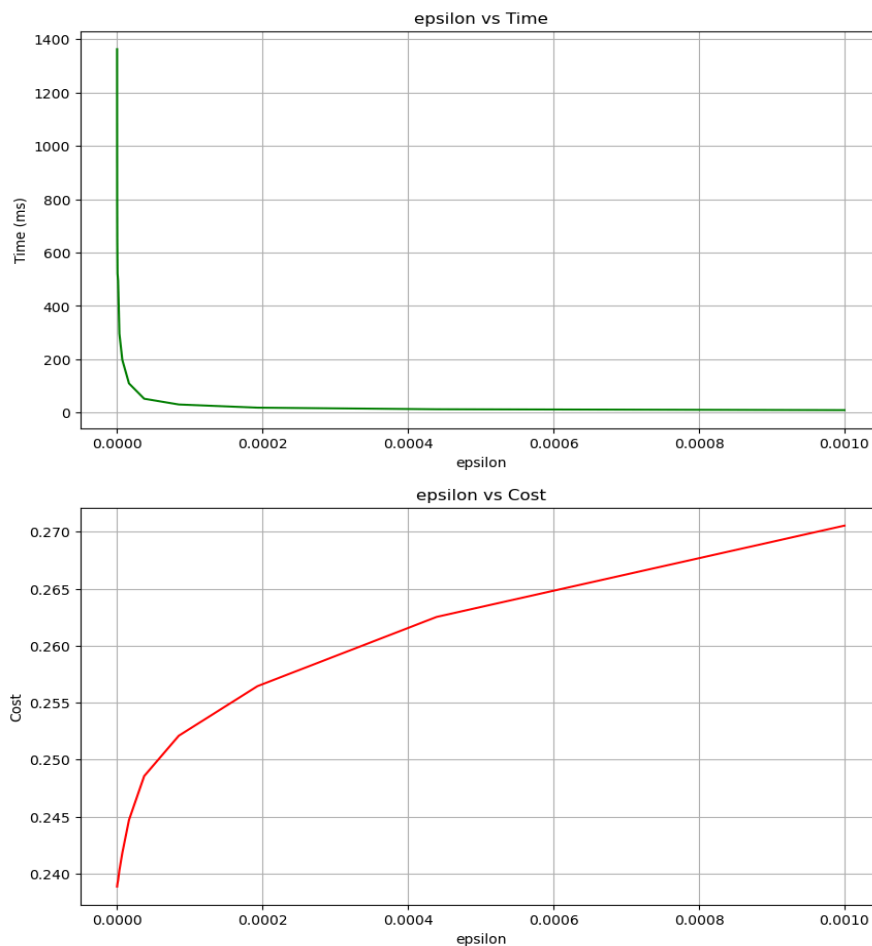
Although, we should have observed that **small alpha values have low cost and high execution time**, the plot **contradicts** this observation. When alpha is **too small**, the steps of the algorithm are also **very small** to the point where **the difference between the cost before and**

*after taking that step is not big enough (less than the convergence threshold) which tricks the algorithm into thinking that it is converging **even though it's still very far away from the minimum**. Which results in **quick execution (low time)** and **high cost (since it is still far away from the minimal cost)**.*

The chosen value for the learning rate is 3, though any value in $[3, 5]$ is good since both **the cost and the time seem to be close to constant**.

2. Tuning the Convergence Threshold:

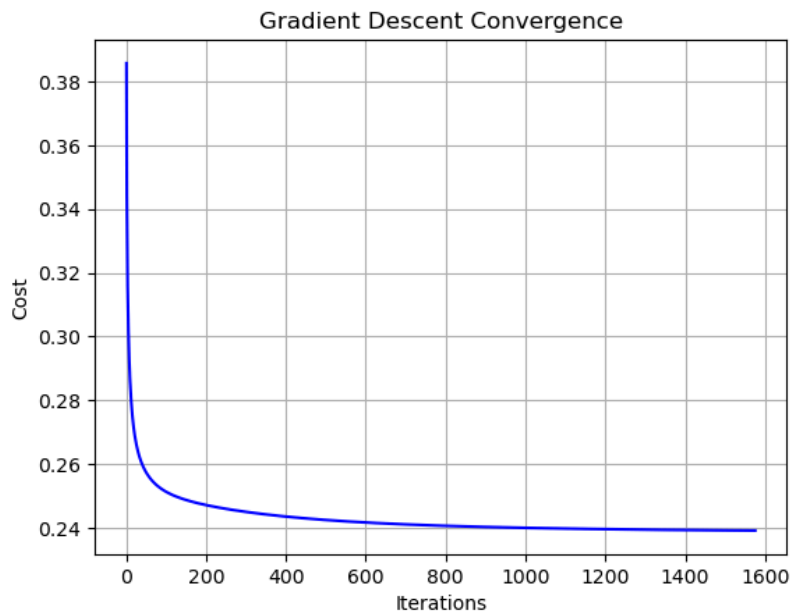
The value of the learning rate is fixed to be 3, and we take the testing values for the convergence threshold in the range $[10^{-3} (0.001), 10^{-8} (0.00000001)]$. The following results were given: *(the detailed values can be found in the appendix B)*



We can see the cost moving quickly **upwards** as we increase the convergence threshold, because by increasing the convergence threshold the algorithm is stopping **further and further away from the actual minimum, worsening the cost**. Since for increasing convergence threshold the algorithm is stopping further away, it is also stopping **earlier** yielding low execution times.

The chosen value for the convergence threshold is $6 \cdot 10^{-7}$ (0.0000006) as it minimizes the cost.

After running gradient descent using the chosen values for α and ϵ , the following plot shows **the convergence of gradient descent** (*that is how it gets closer and closer to the minimum with a decreasing acceleration*)



Initially, the log-loss was **greater than 0.38** for the initial *arbitrarily chosen* values (for this case, zeros). Finally, it improved into **0.24** by the end of the algorithm. We can also see that the

algorithm was first improving quickly and then it started improving **very slowly** *indicating convergence by the last iterations.*

6.4- Logistic Regression Coefficients

After running gradient descent, the **coefficients** that minimized the **log-loss function** (*on the training dataset*), are:

Feature	Corresponding Weight
<i>bias</i>	-3.488
longitude	-4.25
latitude	-4.753
housing_median_age	0.522
bedrooms_per_room	0.419
population_per_household	0.572
rooms_per_household	0.192
log_total_rooms	0.829
log_total_bedrooms	0.07
log_population	-3.514
log_households	2.758
log_median_income	2.196
<1h ocean	0.158
inland	0.415
near ocean	-0.324
island	0.3

For predictors with:

- Positive coefficients: they increase the *log*-odds that make the houses **more likely** to be expensive
- Negative coefficients: they decrease the *log*-odds that make the houses **less likely** to be expensive (*thus more likely to be cheap*)

The magnitude of the coefficients (*their values regardless of the sign*) indicates the **strength** of the influence on the odds (*since the features were scaled, comparing the magnitudes of their coefficients is not meaningless*).

Consequently, the following insights can be made:

- The predictors that make the houses more likely to be **expensive** are (*increase the log-odds*): *housing_median_age, bedrooms_per_room, population_per_household, rooms_per_household, log_total_rooms, log_total_bedrooms, log_households, log_median_income, <1h ocean, inland, island*
- The predictors that make the houses more likely to be **cheap** are (*decrease the log-odds*): *longitude, latitude, log_population, near_ocean*
- The predictors that have the **biggest influence** are: *latitude, longitude, log_population, log_households, log_median_income*

6.5- Assessing the Accuracy

This logistic regression model yielded the following results:

Training log-loss value= 0.239

Testing log-loss value= 0.324

The log-loss value for a model that **blindly** assigns a probability of 0.5 to each class for every observation is around **0.693**. Both the training loss **0.239** and the testing loss **0.324** are **much lower** than 0.693. This indicates that the model has some **strong predictive power**.

However, we can also say that the model is **slightly overfitting** as the difference between the testing loss is **35% higher** than the training loss.

7. Multi-Class Classification

Analysis

In this part, we extend the **binary** classification setting to a **4-class** classification setting, meaning that the target variable will take **4 unique values** instead of 2. These 4 values are: **{‘cheap’, ‘moderate’, ‘expensive’, ‘very expensive’}**

- Houses are considered to be **cheap** if the median value is **less than or equal to \$150,000**
- Houses are considered to be **moderate** if the median value is **greater than \$150,000 and less than or equal to \$250,000**
- Houses are considered to be **expensive** if the median value is **greater than \$250,000 and less than or equal to \$350,000**
- Houses are considered to be **very expensive** if the median value is **greater than \$350,000**

To solve this problem, the following approaches were implemented and used:

7.1- K-Nearest Neighbors

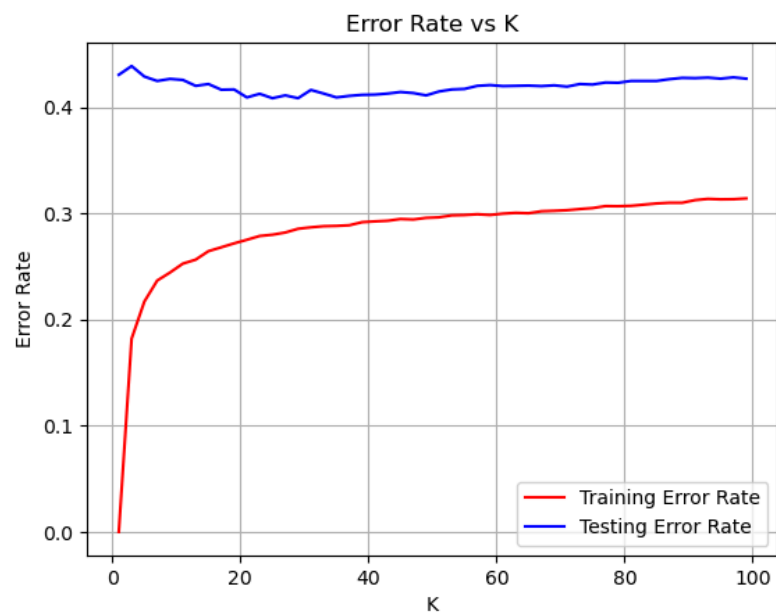
Classification

7.1.1- Theoretical Background

KNN Classification is **very similar** to KNN regression. To classify an observation, the model considers its **k**-closest neighbors and **takes the mode of their targets** (*the most repeated class*). For example *for $k=4$* , and the neighbors of an observation x belong to the following classes: ‘moderate’, ‘expensive’, ‘moderate’, ‘cheap’ then the model would classify x as **moderate** as it is the most repeated class.

7.1.2- Choice of K

Just like KNN regression, we tested different values for k in both the **training and testing sets**, the following plot shows the full process:



Training Error Rate:

Roughly the same behaviour has been already observed and noticed for the KNN regression.

As previously mentioned, for $k=1$ the model **shows severe overfitting**. The error rate keeps increasing (*the model is performing worse and worse*) for higher values of k 's.

Testing Error Rate:

The error rate stays **more than 40% and less than 50%** for all values of k , one may say that 'the error rate *looks constant thus any value of k would give the same performance*', this statement is wrong especially when looking at the detailed values.

From a closer look, we can see that the **optimal range for k falls between 21 and 35** and the error rate takes its minimum of **40.84%** for **$k=25$ and $k=29$** . Thus 25 was the chosen value for k .
(*if the plot is unclear, the detailed error rate for all tested values of k can be found in the appendix B*)

7.2- Generative Classification

Models

7.2.1- Theoretical Background

In previous approaches, we were trying to estimate:

$$P(Y = \text{'class'} \mid X)$$

This is what logistic regression was doing and what KNN classification was also indirectly doing

Now, in generative classification models, they will instead be trying to estimate:

$P(X | Y)$ and $P(Y)$

Roughly speaking, in previous approaches the models were trying to estimate how **the target variable (the classes) are distributed with respect to the predictors**. In GCM, the models are trying to estimate:

- How the **predictors are distributed** inside each class (*for example, in the 'expensive' class, the predictors tend to be like ...*).
- How the classes are **distributed** on their own

The goal throughout the whole classification problem is the same: that is to **classify observations**, so why do we care *about the distribution of the predictors* if the goal is to find the targets? The reason behind this goes back to **Bayes Theorem** that links all three probability distributions together:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)} = \frac{P(X|Y)P(Y)}{\sum_{i=1}^4 P(Y=class_i)P(X|Y=class_i)}$$

So by estimating $P(X | Y)$ and $P(Y)$, we can easily derive an estimation for our main goal that is $P(Y|X)$ using Bayes Theorem.

7.2.2- Estimation of $P(Y)$

$P(Y)$ represents the **distribution of the target variable**. As we know, Y can only take 4 values which are $\{ 'cheap', 'moderate', 'expensive', 'very expensive' \}$, so estimating this distribution is as simple as finding the 4 real numbers **a, b, c, d** that are:

$$P(Y = \text{'cheap'}) = a$$

$$P(Y = \text{'moderate'}) = b$$

$$P(Y = \text{'expensive'}) = c$$

$$P(Y = \text{'very expensive'}) = d$$

As for other values for Y , the probability is equal to 0 (as Y can only take one of the 4 values above)

To estimate these numbers we simply **take the proportions of every class**, which are shown in the table below:

Class	Proportion (Estimation of $P(Y=class)$)
cheap	38%
moderate	36%
expensive	14%
very expensive	12%

These same estimations will be used for all the Generative Classification Models. But what makes each model unique is how **the $P(\mathbf{X} | \mathbf{Y})$** is estimated.

7.2.3- Linear Discriminant Analysis

LDA is a generative classification model. To estimate **$P(\mathbf{X} | \mathbf{Y})$** , two assumptions are made:

- The predictors follow a **multivariate normal distribution** within each class

- These multivariate normal distributions have **the same covariance matrix (across all classes)**, meaning that the difference between the multivariate normal distribution in the 'expensive' class and the 'cheap' class would only be in the mean vector as they must have the same covariance matrix.

In order to perform LDA, we would also need to estimate the **mean vectors** inside each class (*in our case we would have 4 mean vectors*) and **one pooled covariance matrix**.

The estimation of the mean vectors is straightforward, as we estimate it using the mean vectors from our training data. As for the covariance matrix, we estimate it by first **calculating the separate covariance matrices for every class** and then using the **pooled covariance matrix**.

The mean vectors along with the pooled covariance matrix can be found in the appendix B

By applying LDA, we got the following results:

Type of Metric	Value
Training Error Rate	32%
Testing Error Rate	34%

These results are **better than** the results we got in **KNN Classification**, as we have:

- 16.75% drop in the testing error rate

Additionally, the result suggests that the LDA model is **not overfitting** as the difference between the training error rate and the testing error rate is **small**

7.2.4- Quadratic Discriminant Analysis

QDA is another generative classification model that makes the following assumption: **the predictors follow a multivariate normal distribution within each class**. It does **not** restrict the distributions to share the same covariance matrix which allows **more flexibility** as the distributions from one class to another may **differ in both the mean vectors and the covariance matrices**.

By applying QDA, we got the following results:

Type of Metric	Value
Training Error Rate	64%
Testing Error Rate	69%

QDA performed **much worse than both KNN classification and LDA**. One of the common reasons for the **dramatic drop** in performance can be because the features are **highly correlated** (*making the covariance matrices harder to estimate and invert*)

7.2.5- Naive Bayes

Naive Bayes is another **common** generative classification model that makes a *different* assumption than LDA and QDA, that is: **within each class, the predictors are independent.**

Meaning that they do **not** influence each other, one can already expect that this assumption will be **strongly violated** since we have engineered new features and have kept **both the engineered and original features** which will be certainly highly related.

By making this assumption, the estimation of $P(\mathbf{X} | \mathbf{Y})$ becomes simpler, as it will be divided into estimating smaller distributions *that are univariate*, as shown:

$$P(\mathbf{X}|\mathbf{Y}) = \prod_{i=1}^{15} P(X_i|\mathbf{Y})$$

Although the estimation became easier, this is **not enough** to make a complete estimation since we know **nothing about the distributions** (*except that they're independent*) thus we make additional assumptions regarding the distributions:

- Continuous predictors ('longitude', 'latitude', 'housing_median_age', 'bedrooms_per_room', 'population_per_household', 'rooms_per_household', 'log_total_rooms', 'log_total_bedrooms', 'log_population', 'log_households', 'log_median_income') follow a **normal (gaussian) distribution** (*here we are talking about the univariate normal distribution*)
- Binary predictors ('<1h ocean', 'inland', 'near ocean', 'island') follow a **Bernoulli distribution**

Now, what's left is to estimate the means and variances for the predictors following a normal distribution within each class, and the means only (*proportions*) for the predictors following a Bernoulli distribution.

By applying Naive Bayes, we got the following results:

Type of Metric	Value
Training Error Rate	59%
Testing Error Rate	51%

Naive Bayes performed **better than QDA** but still worse than **LDA and KNN classification**.

This poor performance is consistent with the **strong dependence** among the predictors that was observed earlier.

Thus the strongest model overall for this multi-class setting was LDA.

8. Improvements

As this project was *my* first project of this kind, many improvements were **ignored and not implemented**, in order to keep the scope of the project manageable especially that no mature machine learning libraries were used. However, I will still point to some of the potential improvements:

- Addressing multicollinearity: Some of the features were originally **collinear**, and by engineering new features **multicollinearity between features significantly increased**

which led to *unexpected results* in many models such as: Linear Regression and generative classification models.

- Better feature engineering: Although feature engineering **was** used, it was not really as **strong as it could have been**. As *only 3 features* were added.
- Additional models: This project could have made good use of additional models especially in regression, the most obvious example include **non-linear regression models**. It would have been even better to include at least one tree-based model.
- Cross-validation: In this project, the approach that was used to *estimate* the **testing metrics** (*such as the testing MSE and the testing error rate*) was **the validation set approach** (*where the dataset was divided into 2 parts*). However, the more sophisticated method is **cross validation** as it gives the model more data points to train from and also can make **more accurate estimations** regarding the **testing metrics**.
- Applying subset selection: Even though subset selection was implemented in the regression setting, it was **only used** to check which predictors are more or less important. However, the results **were not fed to the actual regression model**.
- Information leakage: In this project, the preprocessing parameters (*like the means and standard deviations of the features*) were calculated using the full dataset before the train/test split, causing information from the test set to leak

9. Conclusion

In this project, I used the [California Housing Prices](#) dataset to predict the **median house values** (*prices*) in a district given some information about it.

The dataset was used in both the **regression** and the **classification settings**, where all of the following models (*alongside their helper algorithms*) were **implemented, used and finally evaluated**: (*the following list includes only the main implementations*)

- Regression: Linear Regression and K-Nearest Neighbors Regression.
- Classification: Binary Logistic Regression, K-Nearest Neighbors Classification, Linear Discriminant Analysis, Quadratic Discriminant Analysis and Naive Bayes.

The use of these models helped reveal **crucial insights** regarding the influence of each feature on the house prices.

10. References

This project made use of:

- [An Introduction to Statistical Learning with Applications in Python](#) by [Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani](#) for statistical learning **theory**.
- [California Housing Prices by Cam Nugent](#) for the dataset's source.

11. Appendices

11.1- Appendix A

Includes **all plots**

All plots can be found [here](#) as **png files**

11.2- Appendix B

Includes **detailed results**

- Statistical summaries for the predictors and the targets variable (*before log-transforming and scaling*)

Metric	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population
count	20640	20640	20640	20640	20640	20640
mean	-119.5697045	35.63186143	28.63948643	2635.763081	536.8388566	1425.476744
std	2.003531724	2.135952397	12.58555761	2181.615252	419.3918779	1132.462122
min	-124.35	32.54	1	2	1	3
25%	-121.8	33.93	18	1447.75	297	787
50%	-118.49	34.26	29	2127	435	1166
75%	-118.01	37.71	37	3148	643.25	1725
max	-114.31	41.95	52	39320	6445	35682

Metric	households	median_income	bedrooms_per_room	population_per_household
count	20640	20640	20640	20640
mean	499.5396802	3.870671003	0.2137940995	3.070655159
std	382.3297528	1.899821718	0.06524816735	10.38604956
min	1	0.4999	0.03715090956	0.6923076923
25%	280	2.5634	0.1752246955	2.429741148
50%	409	3.5348	0.2031591263	2.818115654
75%	605	4.74325	0.240126276	3.282260924
max	6082	15.0001	2.824675325	1243.333333

Metric	<1h ocean	inland	near ocean	island
count	20640	20640	20640	20640
mean	0.4426356589	0.3173934109	0.1287790698	0.000242248062
std	0.4967104649	0.4654732333	0.3349633667	0.01556281184
min	0	0	0	0

25%	0	0	0	0
50%	0	0	0	0
75%	1	1	0	0
max	1	1	1	1

Metric	rooms_per_household	median_house_value (<i>target variable</i>)
count	20640	20640
mean	5.428999742	206855.8169
std	2.474173139	115395.6159
min	0.8461538462	14999
25%	4.440716236	119600
50%	5.229128788	179700
75%	6.052380952	264725
max	141.9090909	500001

- Statistical summaries for the **log-transformed** predictors

Metric	log_total_rooms	log_total_bedrooms	log_population	log_households	log_median_income
count	20640	20640	20640	20640	20640
mean	-1.05E-15	8.26E-17	-9.50E-16	7.71E-16	-1.71E-16
std	1	1	1	1	1
min	-8.707789119	-7.415233064	-7.659496057	-7.281217913	-3.099154187
25%	-0.4678994968	-0.4948911261	-0.4835457544	-0.4765766651	-0.6866334491
50%	0.04474857232	0.03139899777	0.04983313644	0.0432822924	-0.01453691336
75%	0.5672912521	0.5713745401	0.5814146411	0.5809197256	0.6441169692
max	3.933562814	3.756547871	4.695396586	3.754506041	3.500638417

- P-values of the features

feature name	p-value
--------------	---------

intercept	0
longitude	5.87E-107
latitude	4.16E-121
housing_median_age	2.71E-107
bedrooms_per_room	9.62E-12
population_per_household	2.36E-27
rooms_per_household	1.18E-12
log_total_rooms	0.1505186754
log_total_bedrooms	0.0265329055
log_population	0
log_households	1.04E-70
log_median_income	0
<1h ocean	1.53E-05
inland	2.07E-17
near ocean	0.9745418947
island	1.44E-05

- Subset selection using **Mallow's Cp**

step	action	mallow's cp
1	adding log_median_income	7340411565
2	adding inland	5916726743
3	adding housing_median_age	5749606349
4	adding bedrooms_per_room	5612014511
5	adding <1h ocean	5565294148
6	adding log_total_bedrooms	5522058784
7	adding log_population	5108462161
8	adding log_households	5002768074
9	adding population_per_household	4979971614
10	adding rooms_per_household	4967210711
11	adding latitude	4961868854
12	adding longitude	4789526172
13	deleting log_total_bedrooms	4789078330
14	adding island	4785183825
15	adding near ocean	4783576880

- Subset selection using BIC

step	action	BIC
1	adding log_median_income	7343233564
2	adding inland	5921275858
3	adding housing_median_age	5756236963
4	adding bedrooms_per_room	5620643346
5	adding <1h ocean	5575989879
6	adding log_total_bedrooms	5534793333
7	adding log_population	5122205696
8	adding log_households	5018149252
9	adding population_per_household	4997195755
10	adding rooms_per_household	4986298680
11	deleting log_total_bedrooms	4985732982
12	adding latitude	4982265627
13	adding longitude	4809321137
14	deleting <1h ocean	4807697005
15	adding island	4805710819

- The training MSE for every k value *used* in the tuning of **KNN regression**

k	training MSE
1	0
3	0.028616
5	0.046961
7	0.060228
9	0.069825
11	0.07787
13	0.081258
15	0.087572
17	0.092925

19	0.098227
21	0.103446
23	0.108557
25	0.112781
27	0.116506
29	0.120253
31	0.123922
33	0.12748
35	0.131037
37	0.133812
39	0.136664
41	0.139799
43	0.142546
45	0.145164
47	0.147601
49	0.149918
51	0.151995
53	0.154311
55	0.156399
57	0.158705
59	0.160622
61	0.16259
63	0.164815
65	0.166595
67	0.168386
69	0.170276
71	0.171767
73	0.173447
75	0.175253
77	0.176907
79	0.178653
81	0.180239
83	0.181919
85	0.183444

87	0.18516
89	0.186713
91	0.188178
93	0.189659
95	0.190996
97	0.192406
99	0.193753

- The testing MSE for every k value used in the tuning of **KNN regression**

k	testing MSE
1	0.067774
3	0.047614
5	0.04446
7	0.044574
9	0.045013
11	0.04661
13	0.049878
15	0.05202
17	0.05472
19	0.058362
21	0.060754
23	0.063157
25	0.065867
27	0.068574

29	0.071352
31	0.073658
33	0.076338
35	0.078376
37	0.080379
39	0.082782
41	0.084416
43	0.08643
45	0.088072
47	0.089899
49	0.091543
51	0.093139
53	0.094555
55	0.096257
57	0.097963
59	0.099513
61	0.101213
63	0.10292
65	0.104471
67	0.106075
69	0.107381
71	0.108643
73	0.110029
75	0.111574
77	0.113049
79	0.114308
81	0.115767
83	0.117045
85	0.118052
87	0.119362
89	0.120384
91	0.121703
93	0.122908
95	0.123812

97	0.124771
99	0.125949

- Tuning the learning rate parameter in gradient descent

Learning Rate	Cost	Time (ms)	Learning Rate	Cost	Time (ms)	Learning Rate	Cost	Time (ms)	Learning Rate	Cost	Time (ms)
0.001	0.28829	10510.1517	0.0085	0.26109	5175.0197	0.07224	0.24904	2654.5219	0.614	0.24098	1261.1332
0.00104	0.28747	11887.0293	0.00887	0.26074	5198.8417	0.0754	0.24887	2541.2734	0.64085	0.2409	1372.3427
0.00109	0.28666	11759.0464	0.00926	0.2604	5120.6008	0.0787	0.24868	2470.8363	0.66887	0.24082	1390.9627
0.00114	0.28587	11404.1296	0.00966	0.26007	4721.5959	0.08214	0.2485	2349.172	0.69812	0.24075	1310.014
0.00119	0.2851	10970.3843	0.01009	0.25974	5118.4088	0.08573	0.24832	2107.7276	0.72865	0.24068	1206.6568
0.00124	0.28434	11067.333	0.01053	0.25941	4574.0727	0.08948	0.24813	2093.2635	0.76051	0.24061	1086.0509
0.00129	0.2836	11351.3036	0.01099	0.25909	4148.4496	0.09339	0.24794	2002.8162	0.79377	0.24054	1047.6168
0.00135	0.28287	10030.2854	0.01147	0.25878	5022.1654	0.09748	0.24775	1965.125	0.82848	0.24048	1020.6222
0.00141	0.28215	10223.0256	0.01197	0.25847	4473.4161	0.10174	0.24756	1956.0263	0.86471	0.24041	965.1727
0.00147	0.28145	10713.1761	0.01249	0.25816	4847.0076	0.10619	0.24736	2350.6608	0.90252	0.24035	957.5983
0.00153	0.28076	10365.6078	0.01304	0.25786	4438.0808	0.11083	0.24717	2298.9208	0.94199	0.24029	939.7467
0.0016	0.28008	10767.6211	0.01361	0.25757	4106.7348	0.11568	0.24696	2073.8049	0.98318	0.24023	959.4092

0.00167	0.27942	9974.6541	0.01421	0.25728	3701.5066	0.12073	0.246 76	2260.3725	1.02618	0.24018	921.0761
0.00174	0.27877	10310.6979	0.01483	0.25699	4018.9552	0.12601	0.246 55	2452.8268	1.07105	0.24012	926.0617
0.00182	0.27813	9657.598	0.01547	0.25671	4414.7113	0.13152	0.246 34	2300.6236	1.11789	0.24007	913.7743
0.0019	0.2775	9843.1934	0.01615	0.25644	4543.3978	0.13728	0.246 13	2325.689	1.16677	0.24002	905.535
0.00198	0.27688	9290.9139	0.01686	0.25617	4076.2633	0.14328	0.245 92	2220.9427	1.21779	0.23997	969.3245
0.00207	0.27627	9537.5791	0.01759	0.2559	4294.5939	0.14954	0.245 71	2010.6508	1.27104	0.23992	997.3348
0.00216	0.27568	9031.002	0.01836	0.25563	4348.4846	0.15608	0.245 5	2027.2537	1.32663	0.23987	907.6795
0.00226	0.27509	8411.3624	0.01917	0.25538	3965.2722	0.16291	0.245 28	2306.7566	1.38464	0.23983	858.2114
0.00235	0.27452	8486.1496	0.02001	0.25512	3791.7578	0.17003	0.245 08	2277.3764	1.44519	0.23978	833.2072
0.00246	0.27395	8543.7261	0.02088	0.25487	3853.3708	0.17747	0.244 87	2225.9769	1.50838	0.23974	863.9553
0.00256	0.2734	9117.5097	0.02179	0.25463	3571.7534	0.18523	0.244 67	2048.3685	1.57434	0.23969	865.995
0.00268	0.27285	8909.4534	0.02275	0.25439	3237.5935	0.19333	0.244 47	1774.0535	1.64319	0.23965	844.3654
0.00279	0.27232	7199.0893	0.02374	0.25415	2911.4668	0.20178	0.244 28	1806.2654	1.71504	0.23961	805.3232
0.00292	0.27179	6924.224	0.02478	0.25392	3343.7522	0.21061	0.244 09	1818.9724	1.79004	0.23957	837.3459
0.00304	0.27127	6704.6553	0.02586	0.25369	3364.1519	0.21982	0.243 9	1833.8162	1.86832	0.23953	839.947
0.00318	0.27076	7712.4208	0.02699	0.25347	3309.3796	0.22943	0.243 73	1833.5891	1.95002	0.2395	845.4867

0.00331	0.27026	7810.8952	0.02817	0.25325	3205.3242	0.23946	0.243 55	1821.4938	2.03529	0.23946	792.349
0.00346	0.26977	8606.0671	0.02941	0.25303	3001.2143	0.24993	0.243 39	1791.224	2.12429	0.23942	816.407
0.00361	0.26928	6566.4401	0.03069	0.25282	2999.2503	0.26086	0.243 23	2217.1146	2.21719	0.23939	802.9664
0.00377	0.26881	6427.3802	0.03203	0.25261	2749.4551	0.27227	0.243 07	2124.8451	2.31414	0.23935	795.328
0.00393	0.26834	6206.6086	0.03343	0.2524	2940.3177	0.28418	0.242 93	2039.6946	2.41534	0.23932	765.3989
0.00411	0.26788	6125.0724	0.0349	0.2522	3231.4425	0.2966	0.242 78	1878.196	2.52096	0.23929	733.9883
0.00429	0.26743	6151.4048	0.03642	0.252	3321.1546	0.30957	0.242 65	1958.6665	2.6312	0.23928	656.0703
0.00447	0.26698	6515.1822	0.03802	0.2518	3024.6992	0.32311	0.242 51	1768.5197	2.74626	0.23926	623.9435
0.00467	0.26654	6227.6312	0.03968	0.2516	3105.0134	0.33724	0.242 38	1631.9742	2.86635	0.23925	575.3684
0.00487	0.26611	6597.6576	0.04141	0.25141	2492.1672	0.35199	0.242 26	1647.3257	2.99169	0.23925	541.5817
0.00509	0.26569	7335.5416	0.04322	0.25122	2819.0758	0.36738	0.242 14	1710.2042	3.12251	0.23924	516.0768
0.00531	0.26527	6565.4207	0.04511	0.25103	3076.6005	0.38344	0.242 03	1684.1681	3.25906	0.23924	497.9834
0.00554	0.26486	6806.9065	0.04709	0.25085	2831.0395	0.40021	0.241 91	1584.5655	3.40157	0.23923	479.5552
0.00578	0.26446	6501.6743	0.04915	0.25066	2539.2457	0.41771	0.241 81	1773.9836	3.55032	0.23922	453.311
0.00604	0.26406	6551.4865	0.0513	0.25048	2336.8725	0.43598	0.241 7	1784.8301	3.70557	0.23922	423.4575
0.0063	0.26367	5351.4341	0.05354	0.2503	2326.534	0.45504	0.241 6	1494.0711	3.86762	0.23921	403.8372

0.00657	0.26328	5291.8605	0.05588	0.25012	2375.6528	0.47494	0.241 5	1441.6994	4.03674	0.2392	409.7535
0.00686	0.2629	4859.3953	0.05832	0.24994	2549.3107	0.49571	0.241 41	1490.9344	4.21327	0.23919	382.3394
0.00716	0.26253	4931.5544	0.06087	0.24976	2568.4992	0.51739	0.241 32	1438.5703	4.39751	0.23918	375.9898
0.00748	0.26216	5048.9199	0.06354	0.24958	2552.395	0.54001	0.241 23	1433.4633	4.58981	0.23917	363.465
0.0078	0.2618	5040.3616	0.06631	0.2494	2566.4251	0.56363	0.241 4	1297.3976	4.79052	0.23916	361.1025
0.00814	0.26144	4777.9168	0.06921	0.24922	2738.554	0.58828	0.241 06	1256.348	5	0.23915	366.1941

- Tuning the convergence threshold parameter in gradient descent

Convergence Threshold	Cost	Time (ms)
1.00E-03	0.27054	9.394
4.39E-04	0.26253	12.6228
1.93E-04	0.25645	18.6083
8.48E-05	0.2521	30.4836
3.73E-05	0.24856	51.9789
1.64E-05	0.2447	109.4901
7.20E-06	0.24181	197.7547
3.16E-06	0.24028	296.2052
1.39E-06	0.23941	493.19
6.11E-07	0.23915	521.917
2.68E-07	0.239	642.3198
1.18E-07	0.23892	785.1057
5.18E-08	0.23888	1016.545
2.28E-08	0.23887	1187.6592

1.00E-08	0.23886	1362.5901
----------	---------	-----------

- The training error rate for every k value used in the tuning of **KNN classification**

k	Training Error Rate	k	Training Error Rate
1	0	51	0.29621
3	0.18193	53	0.29809
5	0.21699	55	0.29839
7	0.23668	57	0.29912
9	0.24425	59	0.29845
11	0.2526	61	0.29972
13	0.25636	63	0.30045
15	0.26435	65	0.30008
17	0.26799	67	0.30196
19	0.27168	69	0.30239
21	0.27507	71	0.30293
23	0.27865	73	0.30402
25	0.27986	75	0.30493
27	0.28192	77	0.30681
29	0.28543	79	0.30669
31	0.28682	81	0.30705
33	0.28785	83	0.3082
35	0.28815	85	0.30929
37	0.28876	87	0.30996
39	0.29167	89	0.30996
41	0.29239	91	0.31244
43	0.293	93	0.31359
45	0.29463	95	0.31317
47	0.29421	97	0.31329
49	0.29572	99	0.31401

- The testing error rate for every k value used in the tuning of **KNN classification**

k	Testing Error Rate	k	Testing Error Rate
1	0.43047	51	0.41473
3	0.43871	53	0.41667
5	0.42878	55	0.41715
7	0.42466	57	0.42006
9	0.4266	59	0.42078
11	0.42563	61	0.41982
13	0.42006	63	0.42006
15	0.42175	65	0.4203
17	0.41642	67	0.41982
19	0.41667	69	0.42054
21	0.40916	71	0.41933
23	0.41255	73	0.42175
25	0.40843	75	0.42127
27	0.41109	77	0.42321
29	0.40843	79	0.42297
31	0.41618	81	0.42466
33	0.41279	83	0.42466
35	0.40916	85	0.42466
37	0.41061	87	0.42636
39	0.41158	89	0.42757
41	0.41182	91	0.42733
43	0.41279	93	0.42781
45	0.41424	95	0.42684
47	0.41328	97	0.42805
49	0.41109	99	0.42684

- The prior probabilities used in GCM

Class	Prior probability estimation
-------	------------------------------

cheap	0.38311531007751937
moderate	0.3616157945736434
expensive	0.13771802325581395
very expensive	0.11755087209302326

- The pooled covariance matrix used in LDA

9.13E-01	-8.61E-01	-1.72E-01	5.14E-02	8.93E-03	-1.87E-02	4.13E-02	6.12E-02	1.12E-01	5.76E-02	2.87E-02	1.78E-01	-3.29E-02	4.28E-02	1.31E-04
-8.61E-01	9.06E-01	6.80E-02	-1.16E-01	-5.21E-03	1.33E-01	-5.25E-03	-4.99E-02	-1.32E-01	-6.86E-02	2.67E-03	-2.09E-01	1.18E-01	-6.19E-02	-2.21E-04
-1.72E-01	6.80E-02	1.02E+00	1.68E-01	7.05E-03	-1.89E-01	-3.44E-01	-2.87E-01	-2.41E-01	-2.55E-01	-2.14E-01	2.19E-02	-1.05E-01	-2.06E-03	2.49E-04
5.14E-02	-1.16E-01	1.68E-01	1.01E+00	-4.91E-03	-3.58E-01	-2.31E-01	7.82E-02	-1.70E-02	1.99E-02	-4.70E-01	7.01E-02	-1.01E-01	2.06E-02	3.75E-04
8.93E-03	-5.21E-03	7.05E-03	-4.91E-03	2.44E-01	-2.96E-04	-3.90E-02	-4.21E-02	2.44E-02	-4.02E-02	1.22E-02	5.75E-03	-1.10E-04	-2.39E-03	-5.12E-06
-1.87E-02	1.33E-01	-1.89E-01	-3.58E-01	-2.96E-04	1.13E+00	1.26E-01	-9.93E-03	-1.81E-01	-1.82E-01	2.49E-01	-7.41E-02	1.08E-01	-1.72E-02	-4.63E-05
4.13E-02	-5.25E-03	-3.44E-01	-2.31E-01	-3.90E-02	1.26E-01	9.64E-01	9.19E-01	8.45E-01	9.01E-01	1.61E-01	-2.65E-02	4.58E-02	-2.49E-03	-2.19E-04
6.12E-02	-4.99E-02	-2.87E-01	7.82E-02	-4.21E-02	-9.93E-03	9.19E-01	9.84E-01	8.74E-01	9.48E-01	-2.22E-02	-7.28E-04	7.23E-03	5.90E-03	-6.44E-05
1.12E-01	-1.32E-01	-2.41E-01	-1.70E-02	2.44E-02	-1.81E-01	8.45E-01	8.74E-01	9.77E-01	9.13E-01	3.15E-02	4.73E-02	-2.03E-02	-3.51E-03	-2.26E-04
5.76E-02	-6.86E-02	-2.55E-01	1.99E-02	-4.02E-02	-1.82E-01	9.01E-01	9.48E-01	9.13E-01	9.75E-01	-5.27E-03	1.18E-02	-8.30E-03	6.41E-03	-2.11E-04
2.87E-02	2.67E-03	-2.14E-01	-4.70E-01	1.22E-02	2.49E-01	1.61E-01	-2.22E-02	3.15E-02	-5.27E-03	6.02E-01	-9.88E-03	4.41E-02	-1.60E-02	-4.48E-04
1.78E-01	-2.09E-01	2.19E-02	7.01E-02	5.75E-03	-7.41E-02	-2.65E-02	-7.28E-04	4.73E-02	1.18E-02	-9.88E-03	2.15E-01	-1.03E-01	-4.89E-02	-1.77E-04
-3.29E-02	1.18E-01	-1.05E-01	-1.01E-01	-1.10E-04	1.08E-01	4.58E-02	7.23E-03	-2.03E-02	-8.30E-03	4.41E-02	-1.03E-01	1.61E-01	-3.09E-02	-2.14E-05
4.28E-02	-6.19E-02	-2.06E-03	2.06E-02	-2.39E-03	-1.72E-02	-2.49E-03	5.90E-03	-3.51E-03	6.41E-03	-1.60E-02	-4.89E-02	-3.09E-02	9.33E-02	-4.34E-05
1.31E-04	-2.21E-04	2.49E-04	3.75E-04	-5.12E-06	-4.63E-05	-2.19E-04	-6.44E-05	-2.26E-04	-2.11E-04	-4.48E-04	-1.77E-04	-2.14E-05	-4.34E-05	3.02E-04

- The mean vectors for every class

targets	longitude	latitude	housing_median_age	bedrooms_per_room	population_per_household	rooms_per_household	log_total_rooms	log_total_bedrooms	log_population	log_households	log_median_income	<1h ocean	inland	near ocean	island
cheap	0.1396	0.1007	-0.0329	0.2256	0.0142	-0.1057	-0.1832	-0.084	-0.0382	-0.1198	-0.7037	0.224	0.617	0.0952	0
moderate	0.3357	-0.3639	0.0134	0.0933	0.0015	-0.0741	0.0163	0.0547	0.1098	0.07	0.0482	0.6083	0.1779	0.0909	0
expensive	0.1706	-0.308	0.0892	-0.1945	-0.0402	0.0909	0.2289	0.1414	0.0453	0.1679	0.5638	0.5976	0.0875	0.1227	0.0009
very expensive	0.1745	-0.3519	0.3927	-0.4169	-0.0554	0.3475	0.2485	0.056	-0.1606	0.0638	1.1513	0.5755	0.0592	0.1571	0.0015